

분산처리

Part 1:

Work Distribution and Scheduling



Computer & Ai

Department of Computer Engineering

Programming for high performance

고성능 병렬 프로그래밍: 반복적인 최적화 과정:

➔ 병렬 프로그램의 **성능 최적화**는 decomposition, assignment, and orchestration 등을 위한 선택을 구체화하는(refining choices) 반복적인 프로세스



1. 분해 (Decomposition): 문제를 병렬로 처리할 수 있는 작은 작업(task)들로 어떻게 나눌 것인가?
2. 할당 (Assignment): 나눠진 작업들을 어떤 프로세서 코어 또는 스레드(일꾼)에게 배분할 것인가?
3. 조율 (Orchestration): 작업 수행 중 필요한 데이터 통신이나 동기화(순서 맞추기)를 어떻게 효율적으로 관리할 것인가?
➔ 성능을 높이기 위해 이러한 단계들에서의 선택을 계속해서 정제(refine)하고 수정하는 과정

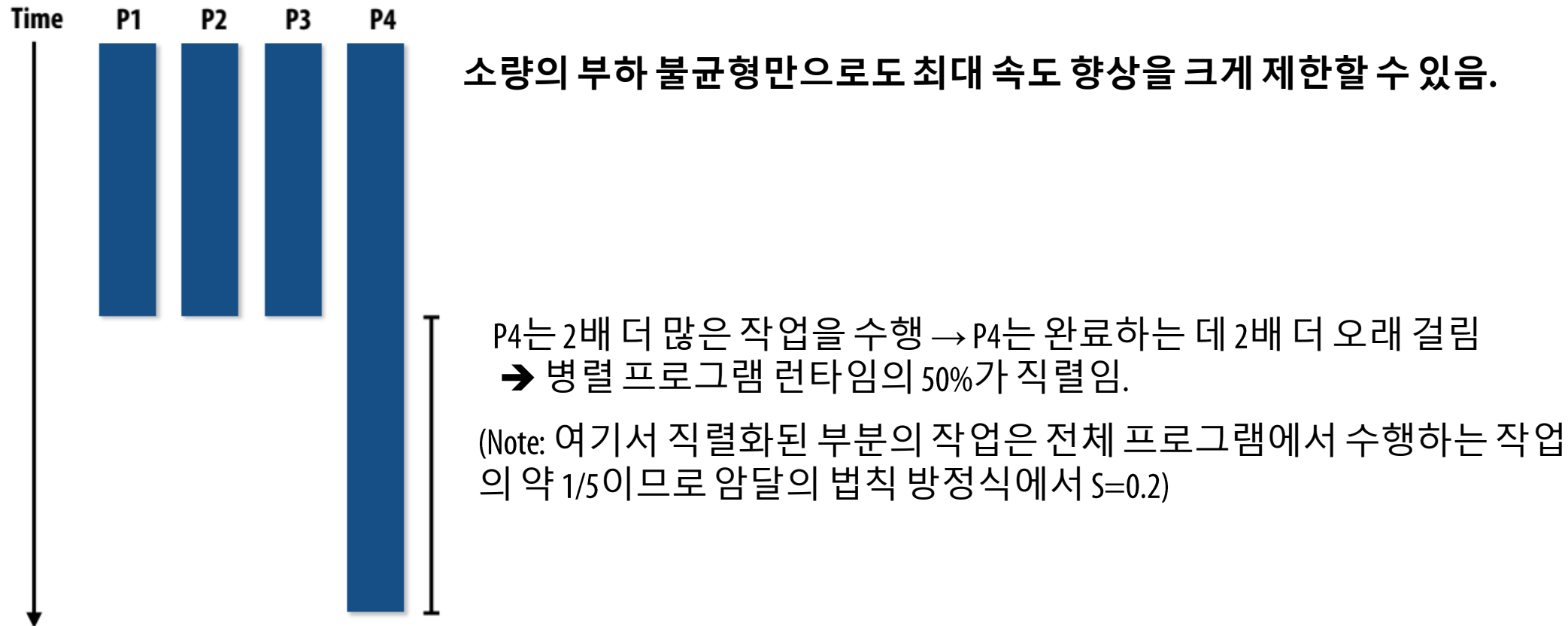
주요 목표(서로 상충되는 목표)

1. 작업 부하 균형 맞추기 (Balance workload): 사용 가능한 실행 리소스에 워크로드 균형 맞추기
 - 목표: 프로그램이 실행되는 동안 모든 프로세서(실행 자원)가 최대한 쉬지 않고 유용한 계산을 하도록 만드는 것. 어떤 프로세서는 할 일이 너무 많아 바쁜데, 다른 프로세서는 일이 없어 노는 상황을 피해야 함. 이상적으로는 모든 프로세서가 동시에 계산을 시작해서 동시에 작업을 마치는 것.
 - 중요성: 아주 약간의 부하 불균형(load imbalance)만 있어도 전체 프로그램 실행 시간이 가장 오래 걸리는 프로세서에 맞춰지기 때문에, 병렬 처리의 효과(speedup)가 크게 제한됨
2. 통신 줄이기 (Reduce communication): 커뮤니케이션 감소(stall 방지)
 - 목표: 스레드나 프로세서 간에 데이터를 주고받는 통신(communication) 양과 빈도를 최소화하는 것
 - 중요성: 통신에는 시간이 걸림. 데이터를 기다리는 동안 프로세서는 아무 계산도 못 하고 멈춰 서게(stall) 될 수 있음
3. 추가 작업(오버헤드) 줄이기 (Reduce extra work (overhead)): 병렬 처리, 할당 관리, 커뮤니케이션 감소 등을 위해 수행되는 추가 작업(오버헤드)을 줄이는 것.
 - 목표: 순수하게 계산 자체에 필요한 작업 외에, 병렬 처리를 위해 추가적으로 수행되는 작업(overhead)**을 최소화하는 것
 - 종류: 이러한 오버헤드에는 병렬성을 높이기 위한 추가 계산, 작업을 스레드에 할당하는 로직(스케줄링), 통신이나 동기화(락, 배리어 등)를 관리하는 데 드는 비용 등이 포함.

팁 #1: 항상 가장 간단한 솔루션을 먼저 구현한 다음,
성능을 측정하여 더 나은 방법이 필요한지 결정.

Balancing the workload

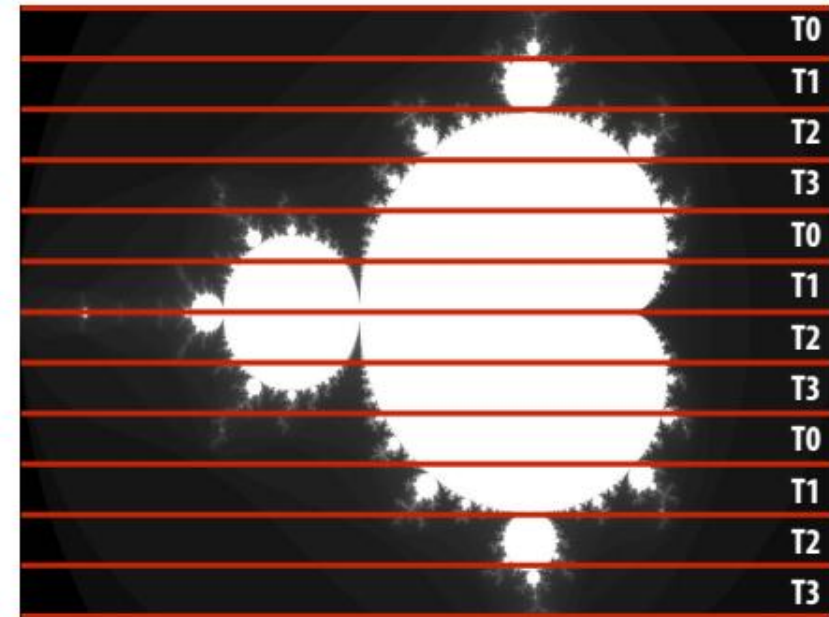
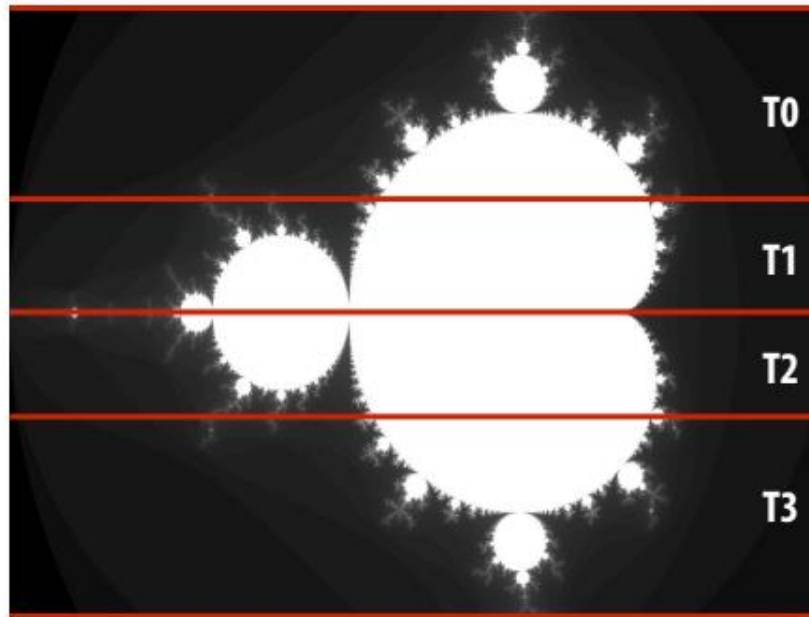
Ideally: 모든 프로세서는 프로그램 실행 중에 항상 컴퓨팅을 수행.
(동시에 계산하고, 동시에 자신의 작업 부분을 처리.)



- 워크로드(workload)는 컴퓨팅 리소스가 작업 완료 또는 결과 도출을 위해 수행하는 처리 작업의 유형과 양을 의미
- 컴퓨터에서 실행하는 모든 애플리케이션이나 프로그램은 워크로드로 간주할 수 있으므로 워크로드는 실행하는 작업의 유형과 수에 따라 크게 달라질 수 있음

Static assignment

- 스레드에 대한 작업 할당은 동적 동작(dynamic behavior)에 의존하지 않음.
 - 작업 할당이 반드시 컴파일 타임(compile-time)에 설정되는 것은 아님(작업량과 작업자 수를 알고 있을 때 할당이 결정되는 것을 정적이라고 함: 입력 데이터 크기, 스레드 수 등과 같은 런타임 파라미터에 따라 작업이 달라질 수 있음).
- 각 스레드에 동일한 수의 그리드 셀 할당 하면?



- 정적 할당의 장점: 할당을 수행하기 위한 런타임 오버헤드가 거의 없는 단순함
(이 예제에서는 할당을 구현하기 위한 추가 할당을 구현하기 위한 추가 작업은 약간의 인덱싱 수학 정도임.)

When is static assignment applicable?

- 정적 할당은 **작업의 비용(실행 시간)과 전체 작업량이 예측 가능할 때** 유용.
 - ➔ 즉, 프로그램을 실행하기 전에 각 작업이 얼마나 걸릴지, 그리고 전체적으로 얼마나 많은 작업을 처리해야 할지 미리 알 수 있을 때 정적 할당을 효과적으로 사용가능.
- 가장 단순 예시) **모든 작업의 비용이 동일하다는 것을 미리 알고 있는 경우.**



- 12개의 동일한 비용을 가진 작업이 있고 4개의 프로세서가 있다면,
- 각 프로세서에 3개씩 작업을 미리 할당할 수 있음.
- 이 경우 각 프로세서가 거의 동시에 작업을 마치게 되어 효율적인 부하 분산(Load Balancing)이 가능.

When is static assignment applicable?

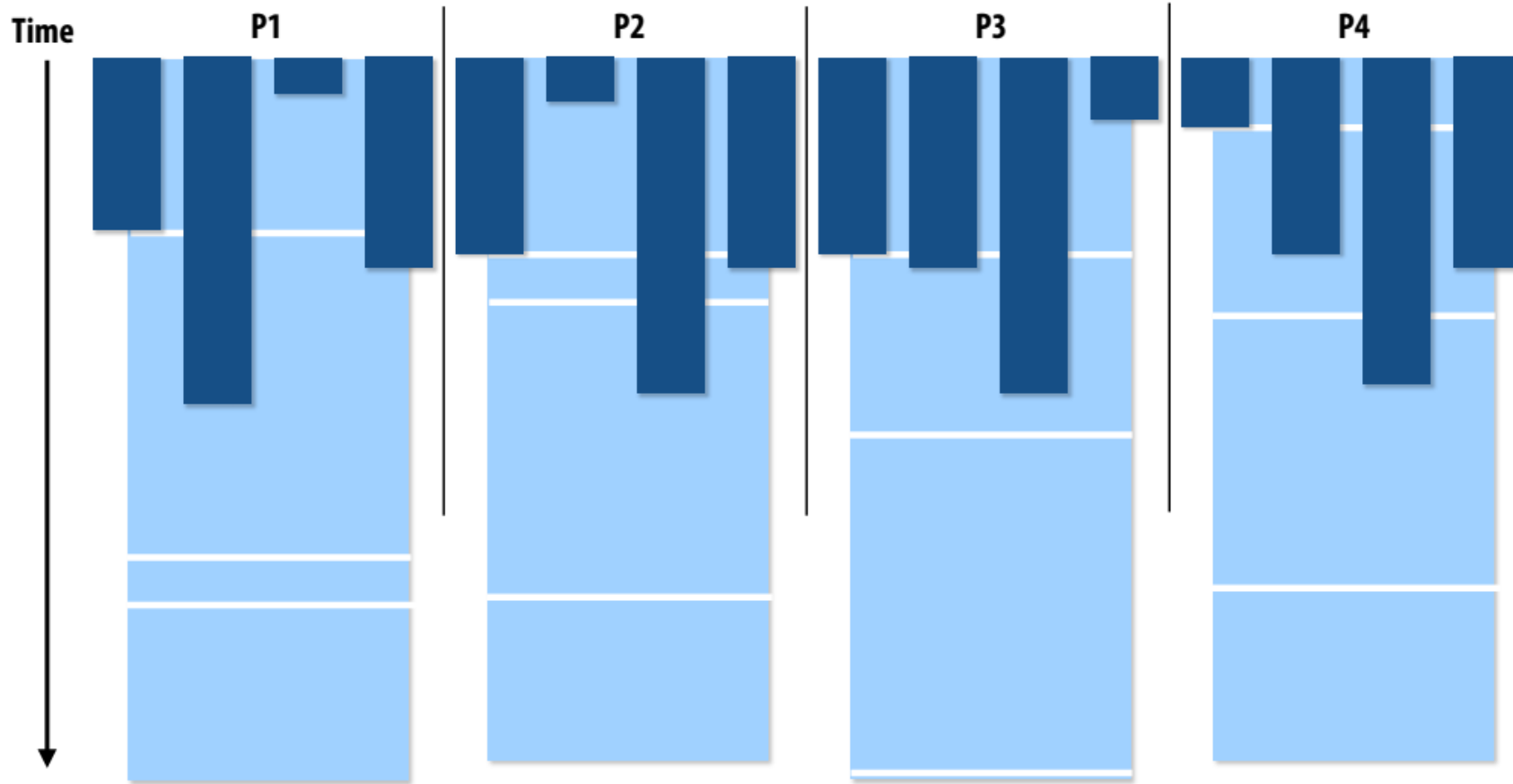
정적 할당은 다음과 같은 경우에 효과적임

1. 전체 작업량과 각 작업의 실행 시간이 미리 예측 가능할 때.
2. 모든 작업의 실행 시간이 동일할 때.
3. 작업 별 실행 시간은 다르지만 그 값이 미리 알려져 있거나 통계적으로 예측 가능할 때.

- 정적 할당은 미리 계산된 할당 방식을 사용하므로, 실행 중에 작업을 분배하는 데 추가적인 오버헤드(Runtime Overhead)가 거의 발생하지 않는다는 장점이 있음.
- 하지만 작업량이나 실행 시간이 예측 불가능하게 변하는 상황에서는 부하 불균형이 심화되어 전체 성능이 저하될 수 있으므로 이런 경우에는 동적 할당(Dynamic Assignment)과 같은 다른 방식을 고려해야 함

정적 할당은 언제 적용되나요?

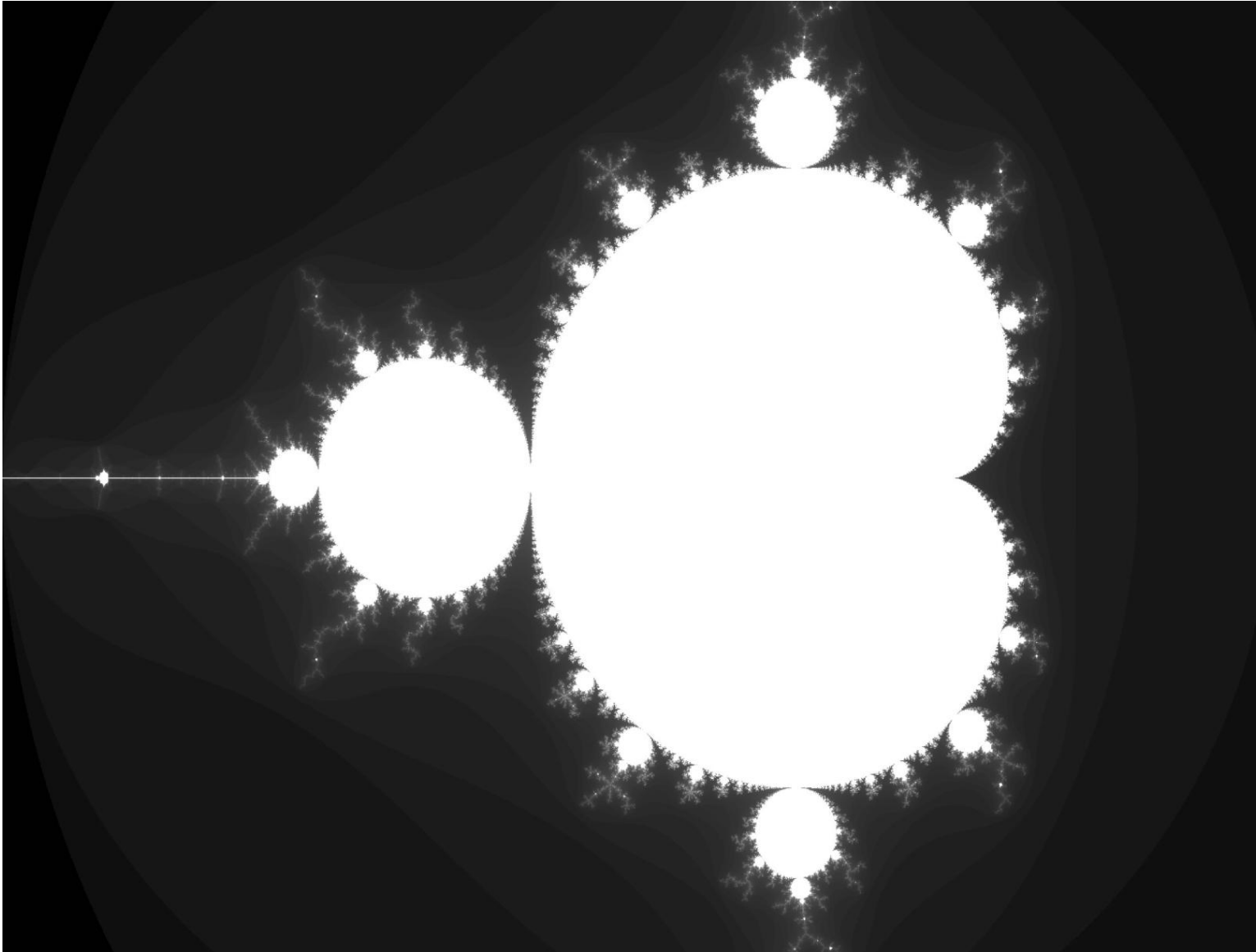
- 작업은 예측 가능하지만 모든 작업의 비용이 동일하지 않은 경우(아래 예시 참조)
- 실행 시간에 대한 통계를 예측할 수 있는 경우(예: 평균 비용이 동일한 경우)



작업에는 균등하지 않지만, 비용을 알고 있는 경우:

➔ 프로세서에 동일한 수의 작업을 할당하여 (평균적으로) 우수한 로드 밸런스를 보장 함.

Example from programming assignment 1 (prog 1)



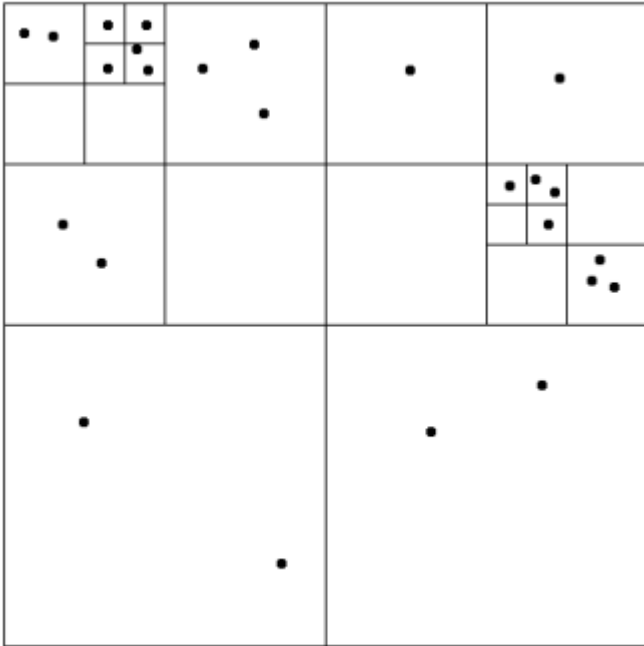
정적 행 인터리브 할당이 이 문제에 대한 좋은 해결책인 이유는 무엇일까요?

<https://gfxcourses.stanford.edu/cs149/fall21/>

<https://github.com/stanford-cs149/asst1>

“Semi-static” assignment

- 가까운 미래에 대한 작업 비용(시간) 예측 가능
 - 아이디어: 최신(recent past)는 가까운 미래를 예측하는 좋은 지표.
- 애플리케이션이 주기적으로 실행을 프로파일링하고 할당을 재조정.
 - 재조정 사이의 간격 동안의 할당은 “정적”임.



파티클 시뮬레이션(Particle simulation):
시뮬레이션이 진행되는 동안 파티클이 움직일 때 작업자에게 파티클을 재분배(동작이 느린 경우 재분배를 자주 수행할 필요가 없음).

컬러 영역은 작업자에게 메시 셀을 할당했음을 나타냄

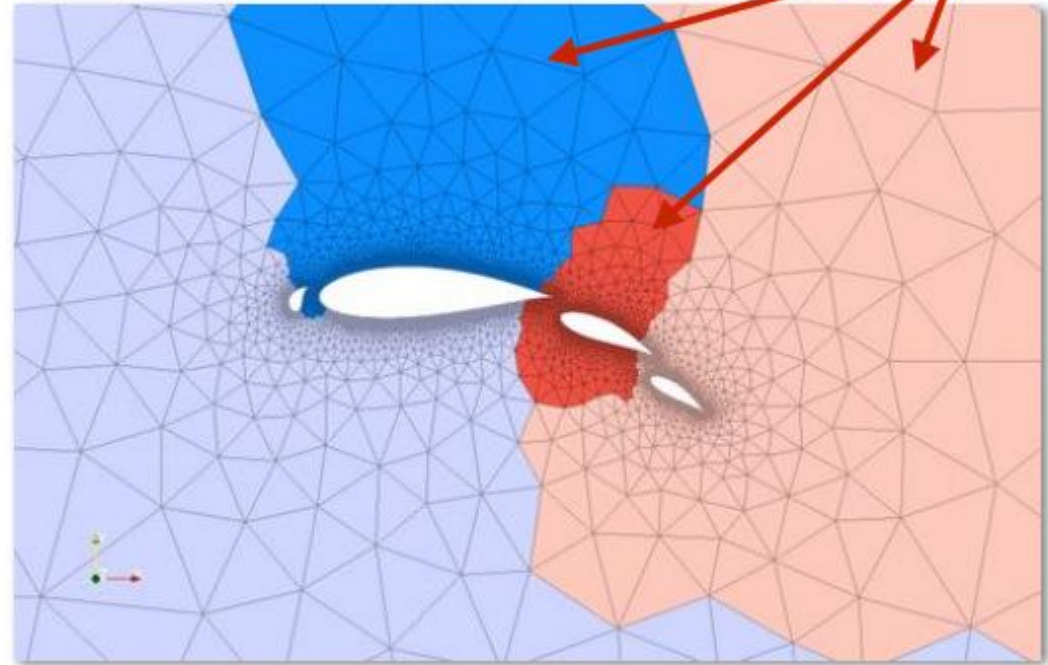


Image credit: <http://typhon.sourceforge.net/spip/spip.php?article22>

적응형 메시(Adaptive mesh):
오브젝트가 이동하거나 #오브젝트가 변경되면 메시가 변경되지만 변경은 느리게 진행 (색상은 메시의 일부가 프로세서에 할당되었음을 나타냄).

Dynamic assignment

- 프로그램이 런타임에 동적으로 할당을 결정하여 부하가 잘 분산되도록 함.

(작업의 실행 시간 또는 총 작업 수를 알 수 없거나 예측할 수 없는 경우).

Sequential program (independent loop iterations)

```
int N = 1024;
int* x = new int[N];
bool* prime = new bool[N];

// assume elements of x initialized here

for (int i=0; i<N; i++)
{
    // unknown execution time
    is_prime[i] = test_primality(x[i]);
}
```

- test_primality(x[i]) 함수를 호출하는 루프가 있는데, 이 소수 판별 함수의 실행 시간은 입력값 x[i]에 따라 크게 달라질 수 있음.
- 즉, 어떤 숫자는 소수 판별에 시간이 오래 걸리고, 어떤 숫자는 금방 판별될 수 있다. 이런 경우 각 반복(iteration)의 실행 시간을 미리 알 수 없기 때문에, 단순히 반복문을 정적으로 쪼개어 스레드에 할당하면 특정 스레드가 다른 스레드보다 훨씬 오래 걸리는 작업들을 맡게 되어 부하 불균형이 발생.

Parallel program (SPMD execution by multiple threads)

```
int N = 1024;

// assume allocations are only executed by 1 thread
int* x = new int[N];
bool* is_prime = new bool[N];

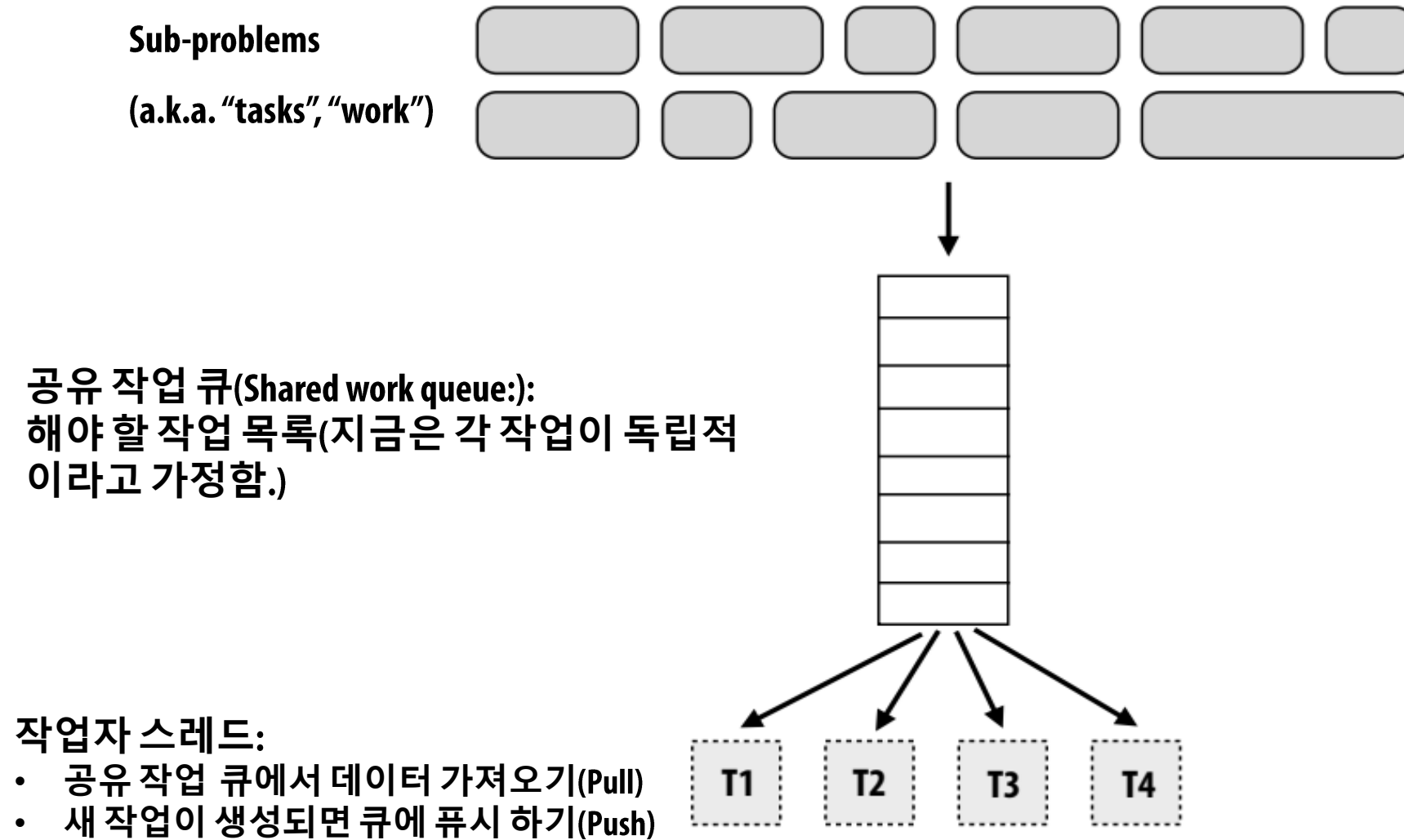
// assume elements of x are initialized here

LOCK counter_lock;
int counter = 0;    // shared variable

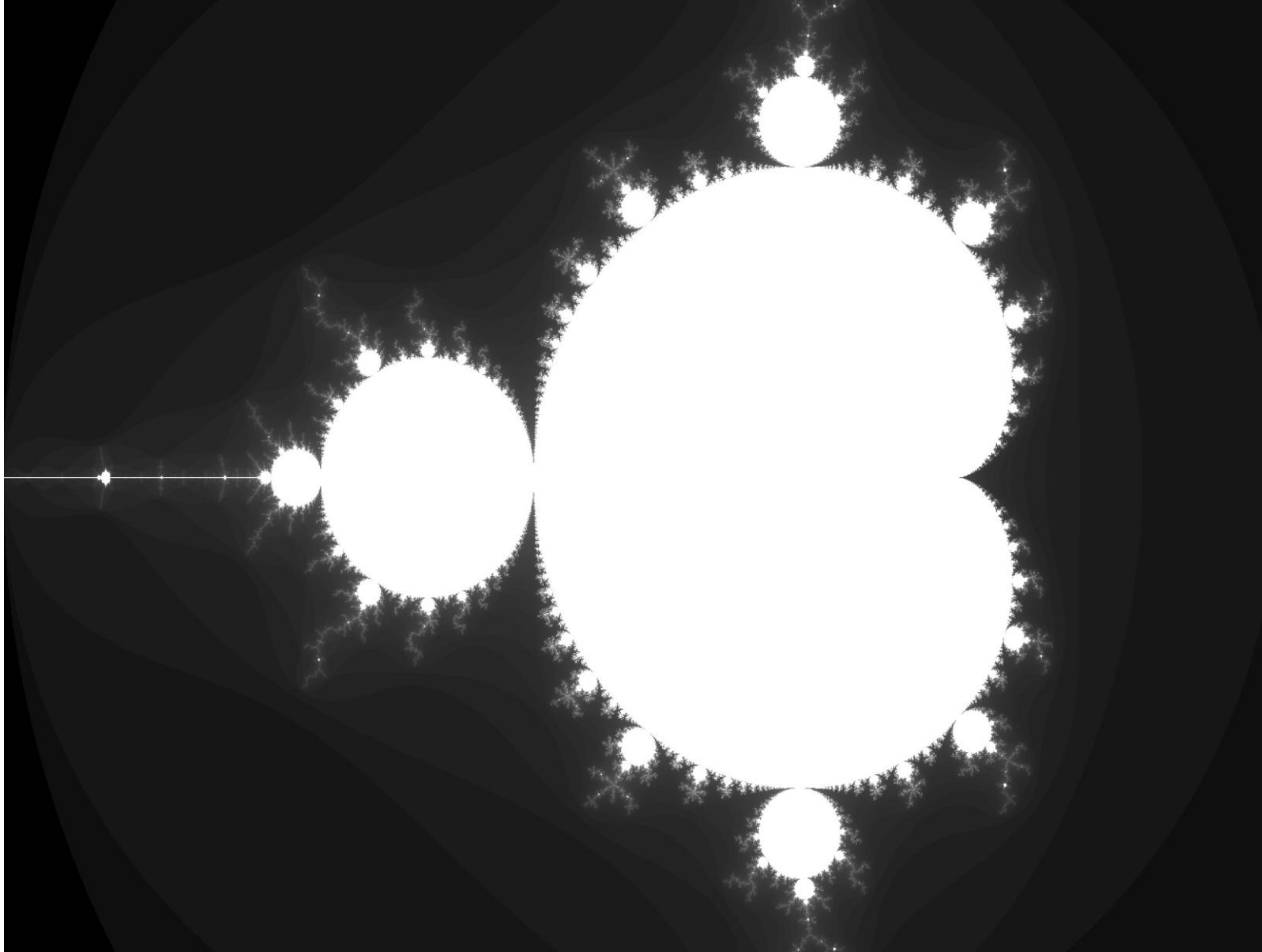
while (1) {
    int i;
    lock(counter_lock);
    i = counter++;
    unlock(counter_lock);
    // atomic_incr(counter);
    if (i >= N)
        break;
    is_prime[i] = test_primality(x[i]);
}
```

- 동적 할당을 위해 공유 변수 counter와 락(lock) 메커니즘(counter_lock)을 사용.
- 여러 스레드가 동시에 counter에 접근하여 다음에 처리할 작업의 인덱스(i)를 가져감.
- 이때 여러 스레드가 동시에 counter를 변경하려 할 수 있으므로 락을 사용하여 한 번에 하나의 스레드만 counter에 접근하도록 하여 데이터의 일관성을 유지.
- 각 스레드는 counter 값을 가져온 후 해당 작업(이 예시에서는 is_prime[i] = test_primality(x[i]))을 수행하고, 모든 작업이 완료될 때까지 이 과정을 반복함.
- 이 방식을 사용하면 작업 시간이 짧은 스레드는 빠르게 다음 작업을 가져가고, 작업 시간이 긴 스레드는 해당 작업을 마칠 때까지 시간이 더 걸리므로, 실행 중에 자연스럽게 부하가 분산됩니다. 즉, 작업의 완료 여부에 따라 동적으로 작업을 할당함으로써 부하 불균형을 완화하는 것

작업 대기열을 사용한 동적 할당



Example from programming assignment 1 (prog 3)



문제를 여러 ISPC 작업으로 나누면 성능이 개선된 이유는 무엇인가요?

What constitutes a piece of work?

이 구현의 잠재적인 성능 문제는 무엇인가요?

```
const int N = 1024;

// assume allocations are only executed by 1 thread
float* x = new float[N];
bool* prime = new bool[N];

// assume elements of x are initialized here

LOCK counter_lock;
int counter = 0;

while (1) {
    int i;
    lock(counter_lock);
    i = counter++;
    unlock(counter_lock);
    if (i >= N)
        break;
    is_prime[i] = test_primality(x[i]);
}
```

가장 간단한 방법으로 루프의 각 반복 한 번, 즉 하나의 요소에 대한 소수 판별 작업(`test_primality(x[i])`)을 '작업 조각(piece of work)' 하나로 보는 경우.
→ 이것을 "Fine granularity partitioning" (세분화된 분할)이라고 함.

장점 Fine granularity partitioning" (세분화된 분할)

- 뛰어난 부하 분산 가능성: 전체 작업의 수가 프로세서(스레드) 수보다 훨씬 많아지므로, 작업 시간이 들쭉날쭉하더라도 유휴 상태인 프로세서가 작업 큐에서 다음 작은 작업을 가져가 처리함으로써 부하 불균형을 효과적으로 줄일 수 있음

단점

- 높은 동기화 비용: 동적 할당에서는 일반적으로 공유 작업 큐에서 작업을 가져오거나 공유 카운터 값을 증가시키는 등의 과정이 필요.
- 이때 여러 스레드가 동시에 접근하지 못하도록 락(Lock)과 같은 동기화 메커니즘을 사용하게 됨.
- 작업 단위가 작으면 각 스레드가 작업을 완료할 때마다 빈번하게 동기화 지점(Critical Section)에 접근해야 하므로, 이로 인한 오버헤드(Overhead)가 커질 수 있음

Time in task 0

Time in critical section

이는 serial program에는 존재하지 않는 오버헤드.

And.. it's serial execution (recall Amdahl's Law)



세분화된 파티셔닝: 1 "작업" = 1 요소

워크로드 밸런스가 양호할 가능성이 높음(작은 작업이 많음)

높은 동기화 비용 발생 가능성

(중요 구간(critical section)에서 직렬화(serialization))

그래서... 문제인가요?

Increasing task granularity(작업 세분성 향상)

```
const int N = 1024;
const int GRANULARITY = 10;

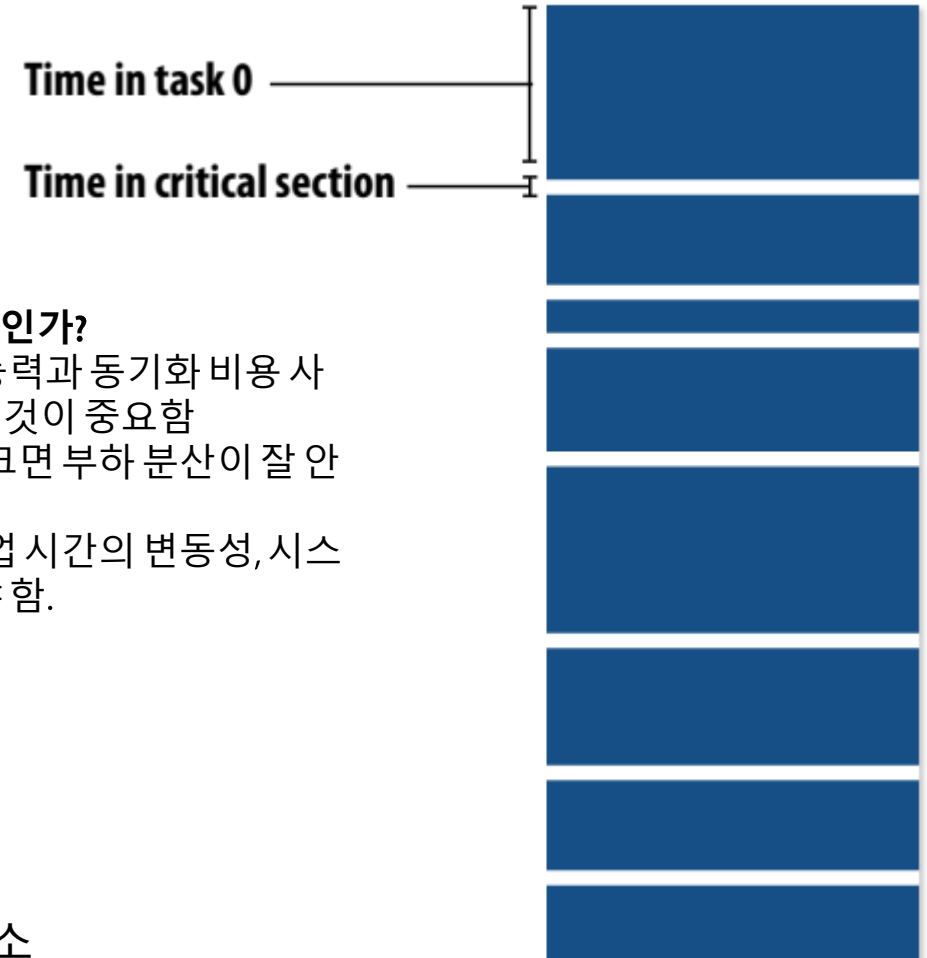
// assume allocations are only executed by 1 thread
float* x = new float[N];
bool* prime = new bool[N];

// assume elements of x are initialized here

LOCK counter_lock;
int counter = 0;

while (1) {
    int i;
    lock(counter_lock);
    i = counter;
    counter += GRANULARITY;
    unlock(counter_lock);
    if (i >= N)
        break;
    int end = min(i + GRANULARITY, N);
    for (int j=i; j<end; j++)
        is_prime[i] = test_primality(x[i]);
}
```

- 병렬 처리를 위해 작업을 어떤 크기로 나눌 것인가?
 - ➔ 이 작업 단위의 크기(Granularity)는 부하 분산 능력과 동기화 비용 사이에 트레이드오프 관계가 있음을 이해하는 것이 중요함
 - ➔ 너무 작으면 동기화 오버헤드가 크고, 너무 크면 부하 분산이 잘 안 될 수 있음.
 - ➔ 이상적인 작업 단위 크기는 문제의 특성, 작업 시간의 변동성, 시스템의 하드웨어 특성 등을 고려하여 결정해야 함.



- 거친 세분성 파티셔닝(Coarse granularity partitioning): 1 “작업” = 10개의 요소
- 동기화 비용 감소
(중요 섹션 입력 10배 감소)

Choosing task size

- 프로세서보다 많은 작업을 보유(부여)할 때 유용.

(작은 작업이 많으면 동적 할당을 통해 워크로드 균형을 맞출 수 있음)

- 세분화된 소규모 작업 (small granularity tasks) 에 동기 부여

- 하지만 할당 관리의 오버헤드를 최소화하기 위해 가능한 한 적은 수의 작업을 원함.

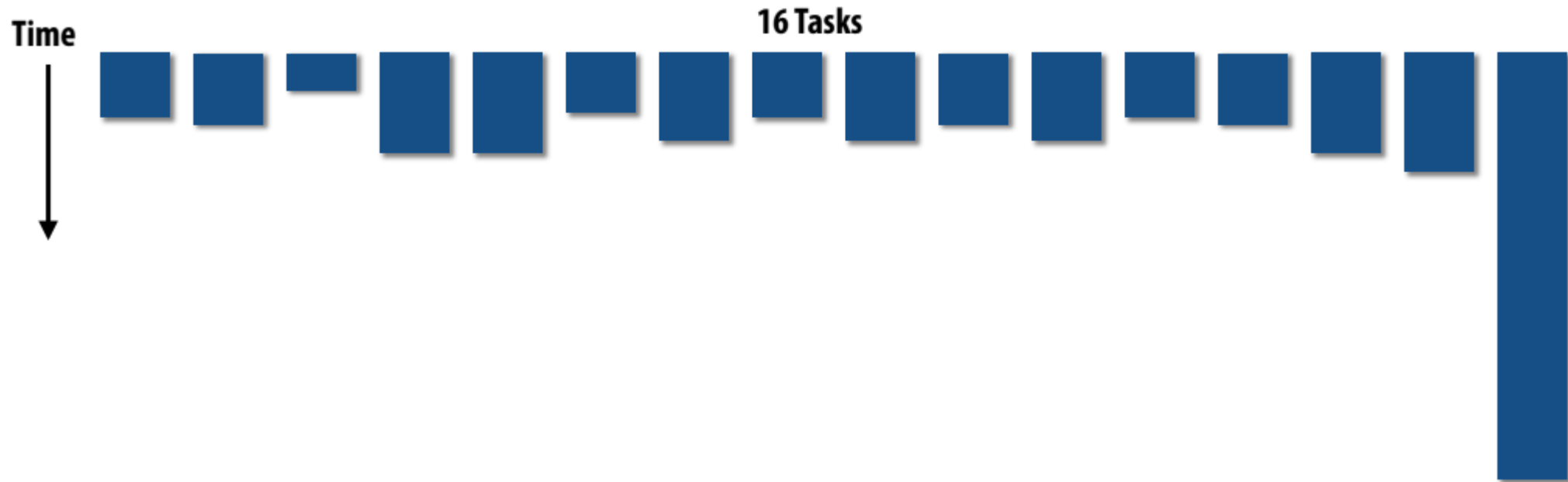
- 큰 세분성 작업 (large granularity tasks)에 동기 부여

- 이상적인 세분성은 여러 요인에 따라 달라짐

(이 과정의 공통 주제: **워크로드와 머신**에 대해 알아야 함)

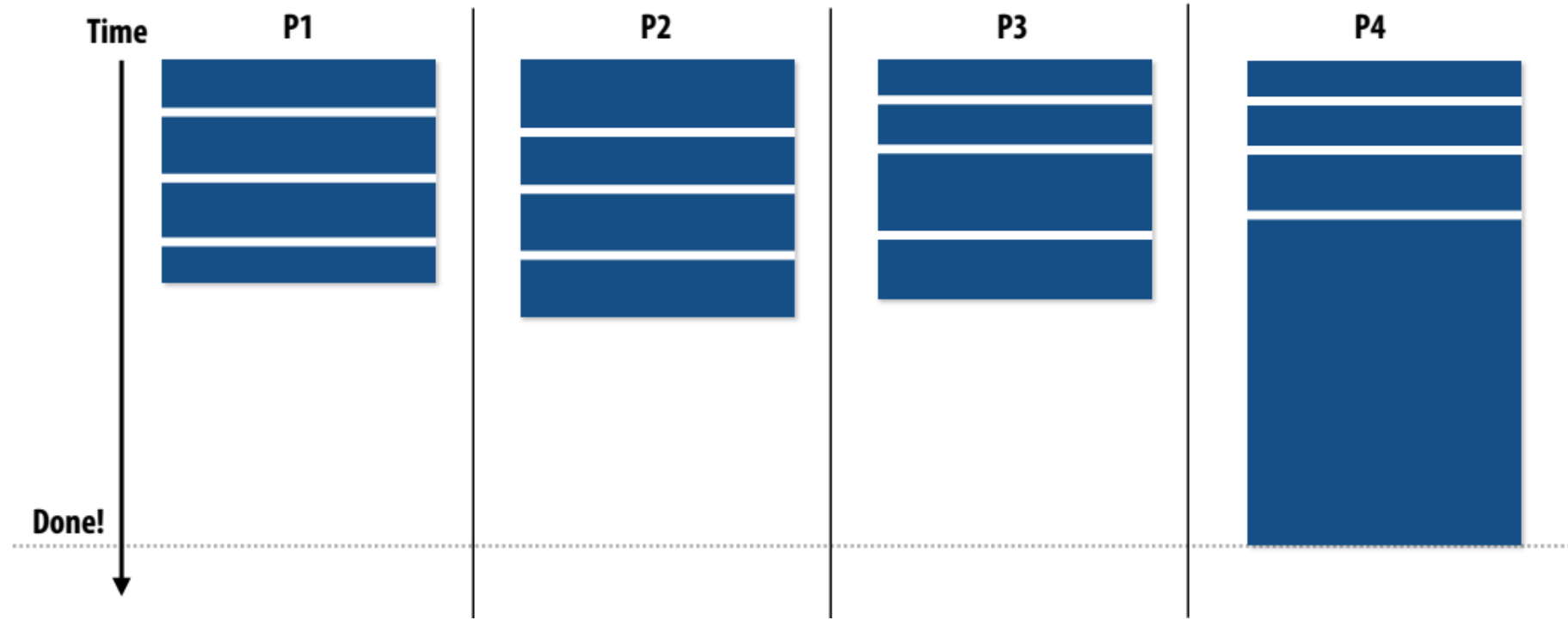
Smarter task scheduling(보다 나은 태스크 스케줄링)

- 공유 작업 대기열(queue 혹은 buffer)을 통한 동적 스케줄링 고려하기
- 시스템에서 이러한 작업을 작업자에게 왼쪽에서 오른쪽 순서로 할당하면 어떻게 될까요?

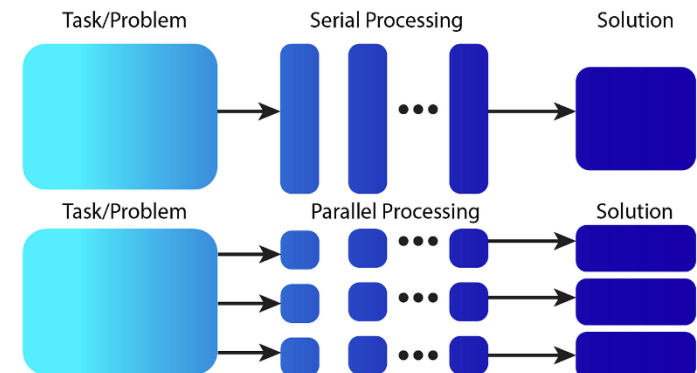


Smarter task scheduling(보다 나은 태스크 스케줄링)

- 스케줄러가 긴 작업을 마지막으로 실행하면 어떻게 되나요? ➔ 로드 불균형이 발생할 수 있습니다!

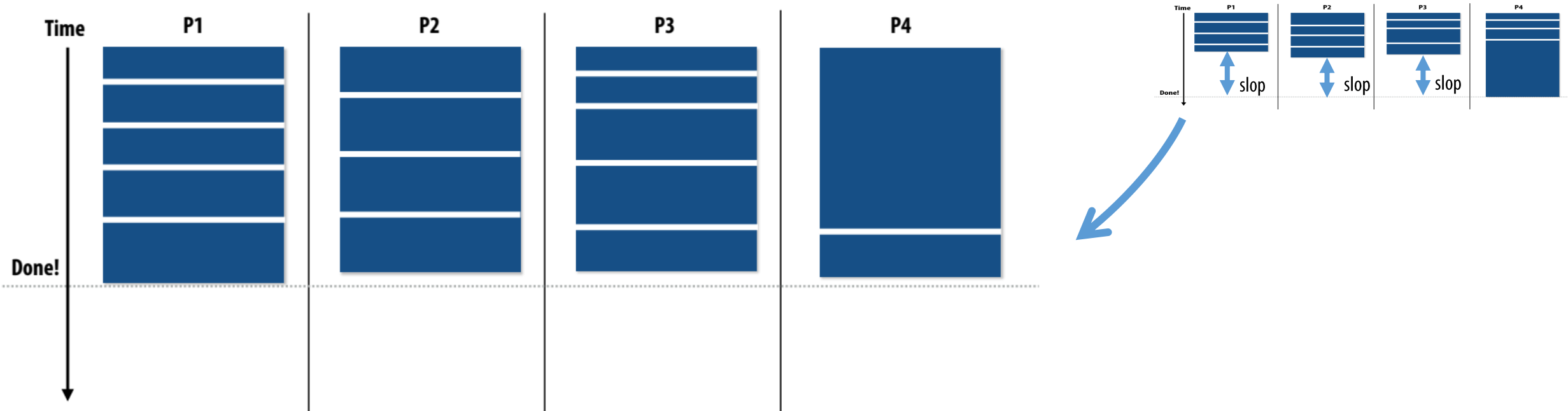


- 불균형 문제에 대한 한 가지 가능한 해결책:
- 작업을 더 많은 수의 작은 작업으로 나누기
 - 이렇게 하면 전체 실행 시간에 비해 “가장 긴 작업(long pole)”이 짧아지기를 바람.
 - 동기화 오버헤드가 증가할 수 있음
 - 불가능할 수도 있음(긴 작업이 근본적으로 순차적일 수도 있음)



Smarter task scheduling(보다 나은 태스크 스케줄링)

- 계산이 끝날 때 “슬로프(slop:오물,찌거기, 낭비시간)”를 줄이기 위해 긴 작업을 먼저 예약하세요.

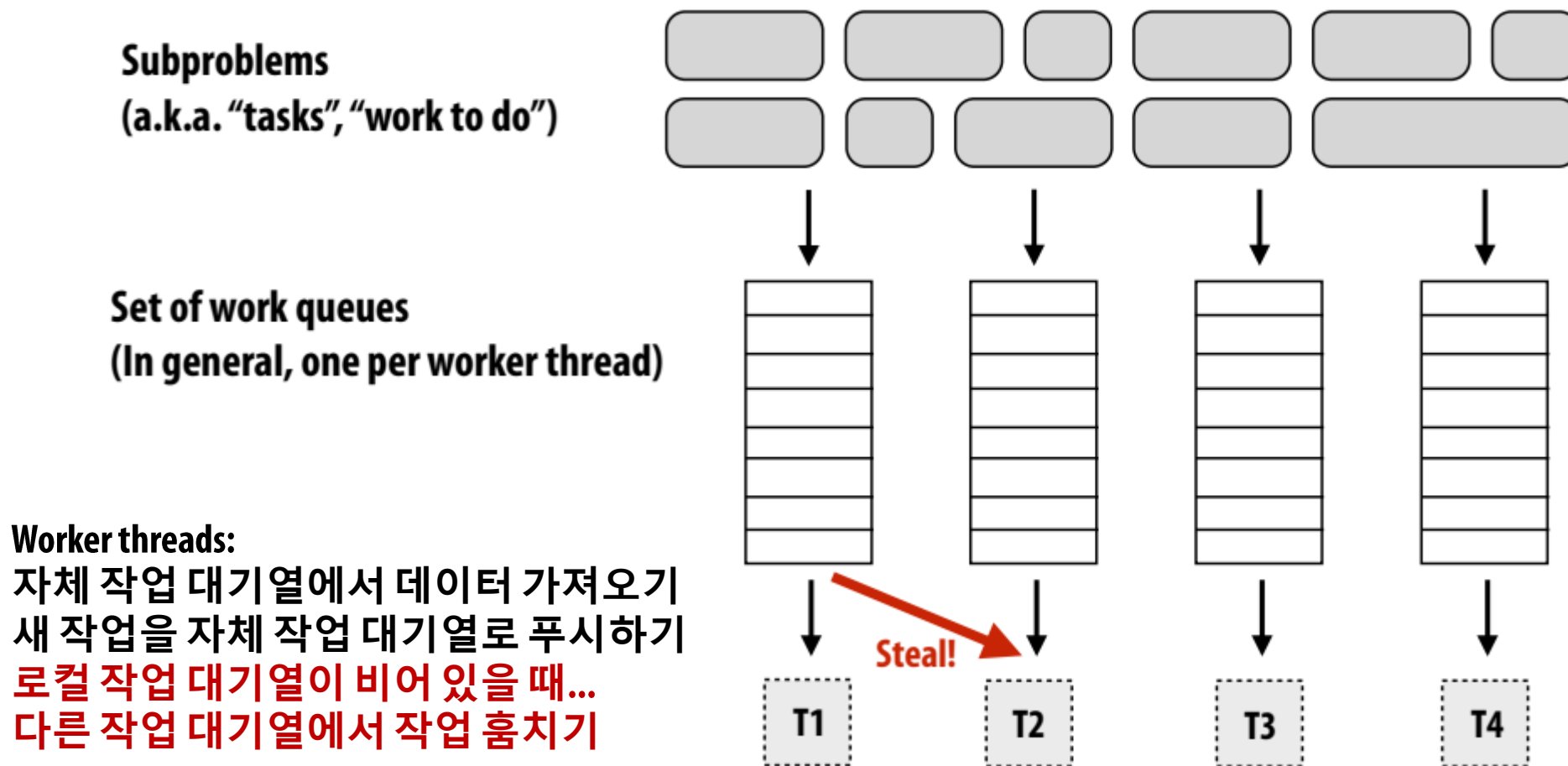


- 또 다른 해결책: 더 스마트한 일정 관리
- 긴 작업을 먼저 스케줄링하세요
 - 긴 작업을 수행하는 스레드는 전체 작업 수는 적지만 다른 스레드와 거의 같은 양의 작업을 수행.
 - 작업량에 대한 어느 정도의 지식이 필요함(비용에 대한 예측 가능성)

Decreasing synchronization overhead using distributed queues

(분산 대기열을 사용하여 동기화 오버헤드 줄이기)

- (모든 작업자가 단일 작업 대기열에서 동기화할 필요 없음)



task queues에서의 작업은 꼭 독립적일 필요는 없음.

...➡ = application-specified
dependency

(애플리케이션 지정 의존성)

•작업 의존성 (Task Dependencies):

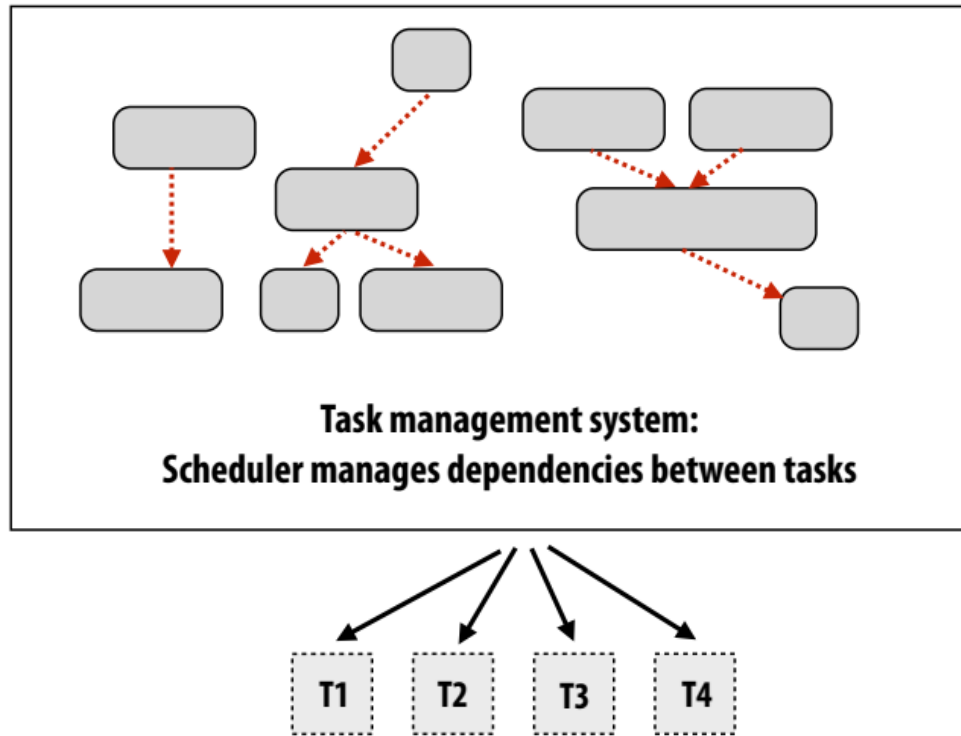
- 특정 작업은 다른 작업이 완료된 후에만 실행가능.
- 이러한 관계를 작업 의존성라고 함. 그림에서 점선 화살표는 이러한 의존성 나타냄

•스케줄러의 역할 (Scheduler's Role):

- 태스크 관리 시스템 내의 스케줄러는 이러한 작업 간의 의존성을 관리.
- 특정 작업에 의존성이 걸려 있다면, 해당 의존성이 충족될 때까지 (즉, 선행 작업이 완료될 때까지) 해당 작업을 워커 스레드(Worker Thread)에 할당하지 않음

- 모든 작업 의존성이 충족될 때까지 task 은 worker thread 에 할당될 수 없음.

- Workers는 new tasks (선택적 명시적 종속성 포함)을 new tasks 에 제출(submit) 할 수 있음.



•명시적 의존성 지정 (Explicit Dependency Specification):

- 프로그래머는 새로운 작업을 태스크 시스템에 제출할 때 명시적으로 의존성을 지정할 수 있음.

`foo_handle = enqueue_task(foo);` - `foo` 작업을 큐에 넣음. 이 작업은 이전에 제출된 다른 작업들과는 독립적.

`bar_handle = enqueue_task(bar, foo_handle);` - `bar` 작업을 큐에 넣음. 이 작업은 `foo` 작업(`foo_handle`로 식별됨)이 완료되기 전까지 실행될 수 없음

Summary

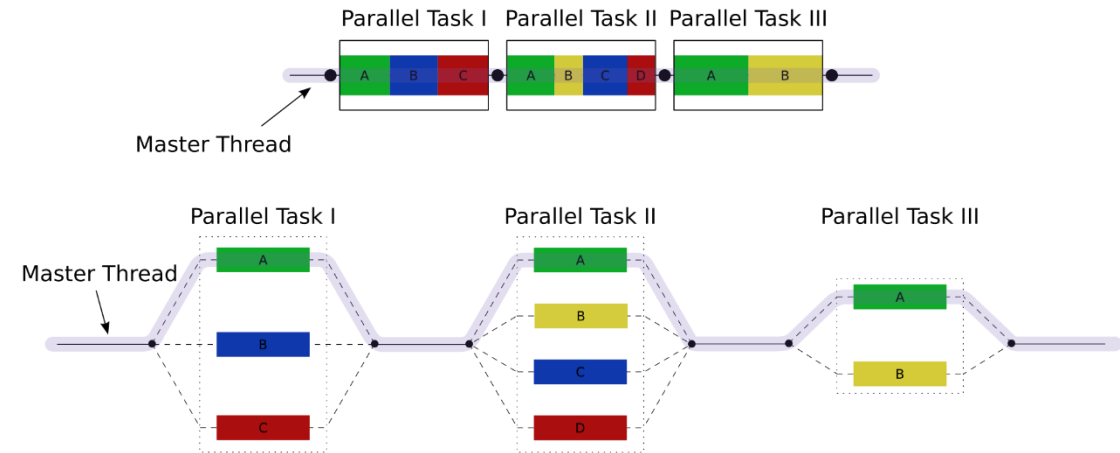
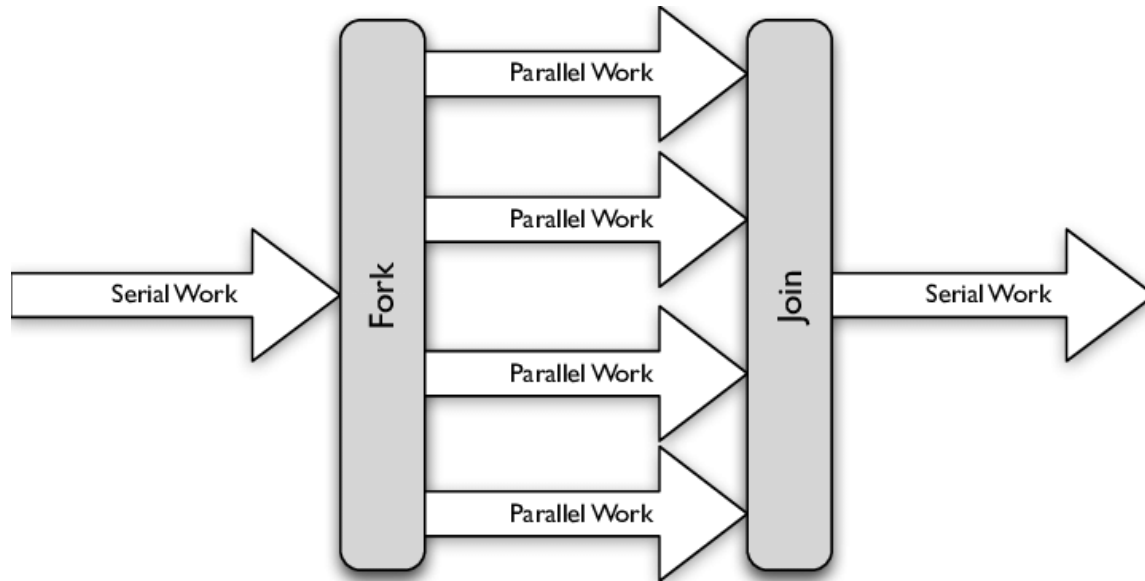
- Challenge : 워크로드(workload balance) 균형 달성하기

- 모든 프로세서가 항상 작동(working)하기를 원함(그렇지 않으면 리소스가 유휴 상태!).
- 그러나 이러한 균형을 달성하기 위한 저비용 솔루션을 원함.
 - ✓ 계산 오버헤드 최소화(예: 스케줄링/할당 로직)
 - ✓ 동기화 비용(synchronization costs) 최소화

- 정적 할당과 동적 할당 비교

- 실제로는 양자택일의 문제가 아니라 연속적인 선택의 연속입니다.
- 워크로드에 대한 사전 지식을 최대한 활용하여 부하 불균형과 작업 관리//동기화 비용을 줄이기
동기화 비용(시스템이 모든 것을 알고 있는 경우에는 완전 정적 할당 사용)

Scheduling fork-join parallelism



Common parallel programming patterns

병렬 프로그램을 작성할 때 자주 사용되는 기본적인 구조 또는 패턴

- 데이터 병렬 처리(Data parallelism):

➔ 많은 데이터 요소에 대해 **동일한 작업 시퀀스** 수행



루프 병렬성 (Loop Parallelism):

```
// ISPC foreach
```

```
foreach (i=0 ... N) {  
    B[i] = foo(A[i]);  
}
```

```
// ISPC bulk task launch
```

```
launch[numTasks] myFooTask(A, B);
```

```
// using higher-order function 'map'
```

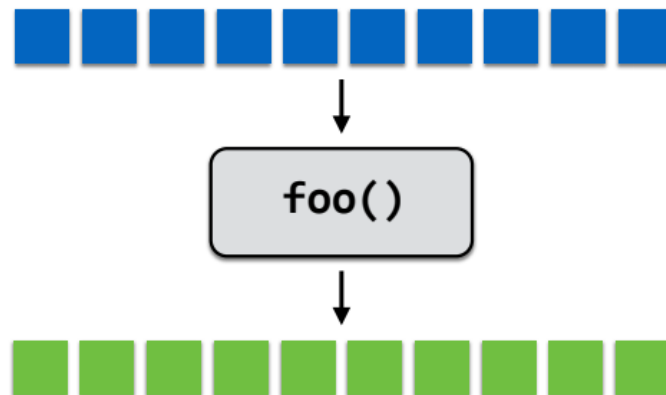
```
map(foo, A, B);
```

```
// openMP parallel for
```

```
#pragma omp parallel for  
for (int i=0; i<N; i++) {  
    B[i] = foo(A[i]);  
}
```

```
// bulk CUDA thread launch (GPU programming, in a future lecture)
```

```
foo<<<numBlocks, threadsPerBlock>>>(A, B);
```



Common parallel programming patterns

- 스레드를 통한 명시적 병렬 처리 관리:
- 실행 단위당(또는 원하는 동시성 양만큼) 하나의 스레드를 생성
 - 아래 예시: C++ 스레드가 있는 C 코드

```
float* A;  
float* B;
```

```
// initialize arrays A and B here
```

```
void myFunction(float* A, float* B { ... }
```

```
std::thread thread[NUM_HW_EXEC_CONTEXTS];
```

```
for (int i=0; i<NUM_HW_EXEC_CONTEXTS; i++) {  
    thread[i] = std::thread(myFunction, A, B);  
}
```

```
for (int i=0; i<num_cores; i++) {  
    thread[i].join();  
}
```



포크-조인 병렬성 (Fork-Join Parallelism):

분할 정복(divide-and-conquer) 알고리즘(예: 퀵 정렬, 병합 정렬)을 병렬화

하나의 작업(부모)이 여러 개의 자식 작업을 생성(fork)하여 병렬로 실행시킨 후, 모든 자식 작업이 완료될 때까지 기다렸다가(join) 다시 실행을 이어가는 방식

Consider divide-and-conquer algorithms

분할정복

정의 : 분할정복 알고리즘은 문제를 나눌 수 없을 때까지 나누어서 각각을 풀면서 다시 합병하여 문제의 답을 얻는 알고리즘이다.

알고리즘을 설계하는 요령

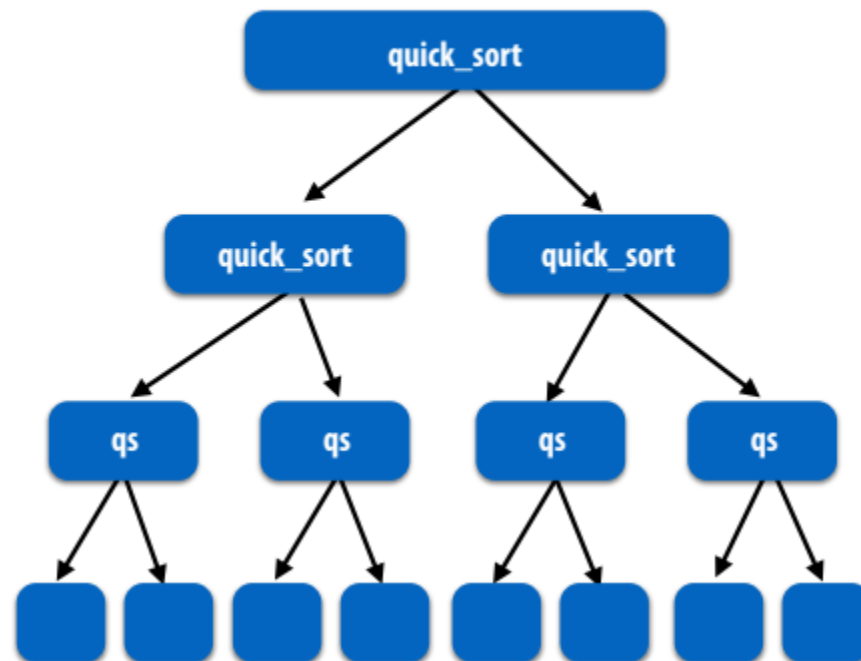
- (1) Divide : 문제가 분할이 가능한 경우, 2개 이상의 문제로 나눈다.
- (2) Conquer : 나누어진 문제가 여전히 분할이 가능하면, 또 다시 Divide를 수행한다. 그렇지 않으면 문제를 푼다.
- (3) Combine : Conquer한 문제들을 통합하여 원래 문제의 답을 얻는다.

Quick sort:

```
// sort elements from 'begin' up to (but not including) 'end'
void quick_sort(int* begin, int* end) {
    if (begin >= end-1)
        return;
    else {
        // choose partition key and partition elements
        // by key, return position of key as 'middle'
        int* middle = partition(begin, end);
        quick_sort(begin, middle);
        quick_sort(middle+1, last);
    }
}
```

independent work!

Dependencies



- 분할 및 정복 알고리즘에 내재된 독립적인 작업을 자연스럽게 표현하는 방법
- 이 강의의 코드 예제는 Cilk Plus에 있습니다.

- C++ 언어 확장

- 원래 MIT에서 개발되어 현재 오픈 표준으로 채택됨(GCC, 인텔 ICC에서)

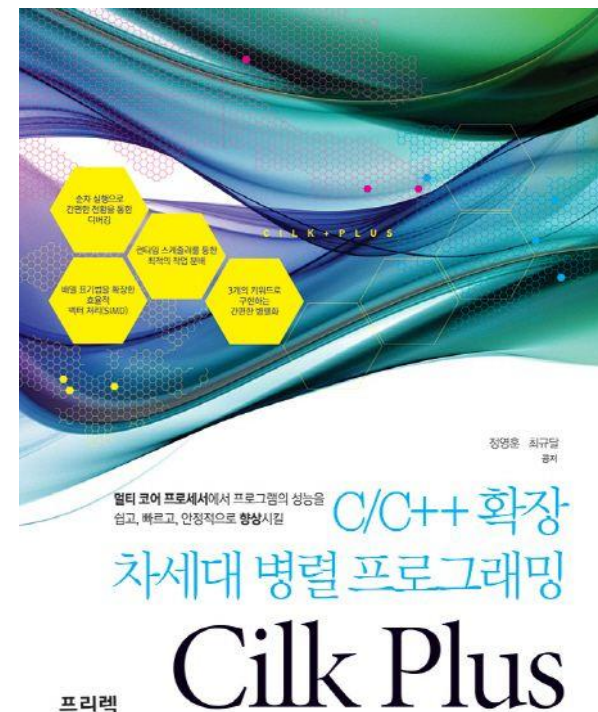
- `cilk_spawn foo(args);` ← **“fork” (create new logical thread of control)**

➤ 의미(Semantics): `foo`를 호출하지만 표준 함수 호출과 달리 호출자는 `foo`의 실행과 함께 비동기적으로 계속 실행할 수 있음.

- `cilk_sync;` ← **“join”**

➤ 의미(Semantics): 현재 함수에 의해 생성된 모든 호출이 완료되면 반환합니다. (스폰된 호출과 “동기화”)

➤ 참고: `cilk_spawn`을 포함하는 모든 함수의 끝에는 암시적 `cilk_sync`가 있음 (의미: Cilk 함수가 반환되면 해당 함수와 관련된 모든 작업이 완료됨).



Fork-join pattern

- Fork/Join의 계산 효율을 극대화하기 위한 원리
- Fork/Join 프레임워크의 계산 효율을 극대화하기 위해 중요한 4가지 원칙이 있음.
 - 병렬성 극대화
 - 오버헤드 최소화
 - 리소스 등의 충돌을 최소화한다.
 - 국소성 극대화

Fork-join pattern

- 적절한 작업 세분화 선택하기

- ✓ Fork/Join 프레임워크에 국한된 이야기일지 모르겠지만, 병렬 처리에서는 흔히 병렬성과 오버헤드가 트레이드 오프 관계가 되기 쉽다.
- ✓ 즉, 태스크의 동시 실행 수를 높게 유지, 즉 병렬성을 높이려고 하면 병렬 처리에 관련된 오버헤드가 증가하고, 반대로 오버헤드를 낮추려고 하면 병렬성이 희생되는 관계이다.

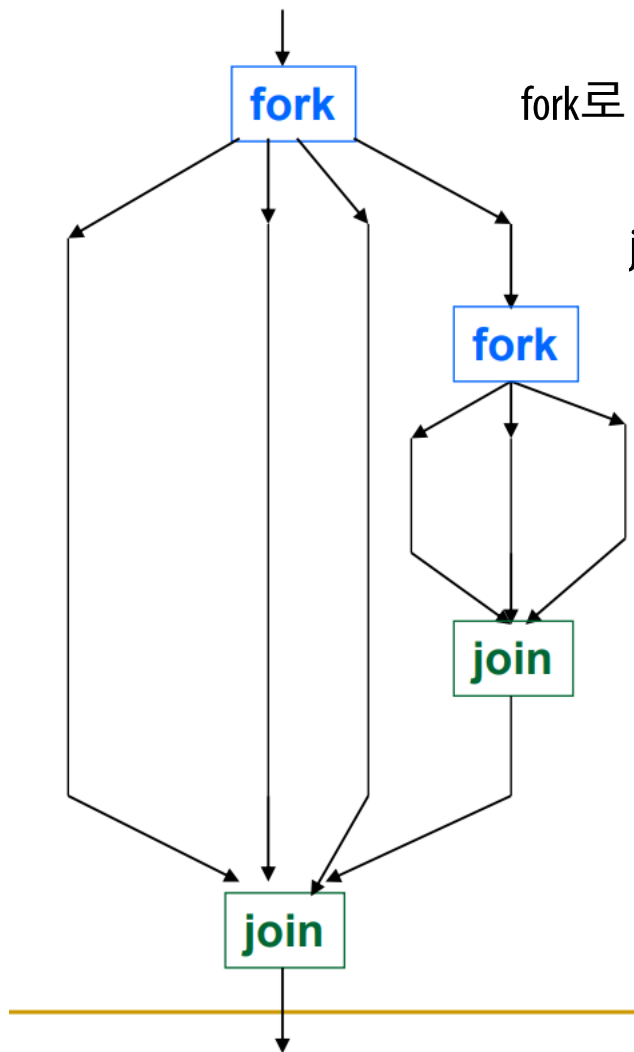
- Fork/Join 프레임워크에서도 이 병렬성과 오버헤드 중 어느 한 쪽에 치우쳐 최적화한다고 해서 전체 계산 효율이 최적화(처리 시간 최소화)되는 것은 아니다.

- ✓ 따라서 병렬성과 오버헤드의 균형을 맞추는 최적화가 필요하다.

Fork-join pattern

- Fork/Join 프레임워크는 divide-and-conquer 과 같이 재귀적으로 태스크를 하위 태스크로 분할하여 처리하는 문제에서 활용하기에 적합하다.
- 이때, 재귀적 분할을 마치고 실제 계산이 이루어질 때의 태스크의 입도가 Fork/Join 프레임워크의 병렬성과 오버헤드의 트레이드오프를 좌우하는 요소가 될 수 있다.
- 여기서 '작업의 세분성'에 대한 이해를 돕기 위해 병합 정렬(Merge Sort)을 예로 들어보자.
 - 병합 정렬은 재귀를 통해 데이터를 최소 단위까지 분할하지 않고, 요소 수가 임계값 이하로 떨어지면 분할을 멈추고 삽입 정렬과 같은 다른 간단한 정렬 알고리즘으로 전환하는 구현을 하기도 한다
 - 이 임계값 설정은 Fork/Join 모델에서 말하는 '적절한 작업의 세분성 선택'에 해당.

Fork-join: 병렬 프로세스의 시작과 종료



fork로 생성된 프로세스(스레드) 간 메모리 공유

join으로 전 프로세스 대기 → 동기화를 하고 있다.

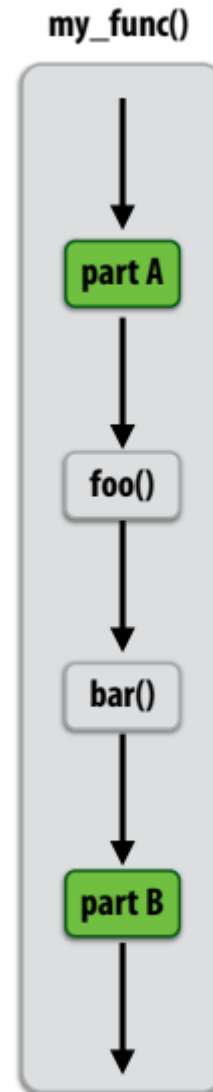
간단한 병렬 프로그램은 fork/join만으로 제어 가능 → OpenMP

Call-return of a function in C*

```
void my_func() {  
    // calling function (part A)  
    foo();  
    bar();  
    // calling function (part B)  
}
```

- 함수 호출의 의미:
 - 제어가 호출된 함수로 이동.
(스레드가 함수에 대한 명령을 실행)
 - 함수가 반환되면, 제어는 호출자에게 다시 반환.
(스레드가 호출자로부터 명령어 실행)

* 그리고 다른 많은 언어



Spawn vs fork

Spawn

- 윈도우즈 및 리눅스/유닉스에서 사용되는 다른 프로세스 생성 방식 중 하나
- 새로운 파이썬 인터프리터를 생성하여 자식 프로세스를 생성하는 방법
- 이 방식은 운영체제에서 별도의 프로세스를 생성하는 방식이므로, 안전성이 높으며 다양한 운영체제에서 사용
- `nlrl2`, `pytorch` 등등에서는 기본적으로 multiprocessing을 spawn 방식으로 합니다.
- 자식 프로세스를 생성할 때, 부모 프로세스의 환경(메모리)을 그대로 복제하지 않음
- 자식 프로세스를 생성할 때, 파이썬 interpreter를 새로 켜고, 돌리려는 타겟이 있는 스크립트를 다시 import 후 타겟 함수 실행
- 그러므로 각각의 프로세스는 독립적인 메모리 공간을 가지게 됩니다.

● 장점:

- .이식성: 다양한 운영 체제에서 잘 동작하며, 특히 Windows에서 주로 사용됩니다.
- .안정성: 부모 프로세스의 자원을 복제하지 않고 새로운 프로세스를 완전히 새롭게 시작하기 때문에, 메모리 누수나 상태 공유로 인한 문제를 줄일 수 있음

● 단점:

.시작 시간:

- 새 프로세스를 시작할 때마다 프로그램의 전체 환경을 다시 로드해야 하므로, fork 방식에 비해 상대적으로 시작 시간이 더 길 수 있습니다.

.자원 사용량:

- 새로운 프로세스 환경을 완전히 새롭게 구성해야 하기 때문에, 초기 자원 사용량이 더 클 수 있습니다.

Spawn vs fork

Spawn

- 프로세스가 'spawn' 방식으로 생성될 때, 현재 실행 중인 코드의 상태(예를 들어 전역 변수나 모듈 수준에서 설정된 설정들)가 새 프로세스에 자동으로 전달되지 않음.
- 따라서, 만약 시뮬레이터 스크립트를 실행 중인 코드에서 변경사항이 있더라도, 새로 'spawn'된 프로세스에서는 이러한 변경사항이 적용되지 않은 채로 시작될 수 있음.
- 특히, Pybullet 같은 외부 라이브러리나 시뮬레이션 환경을 사용하는 경우, 코드 내에서 동적으로 변경사항을 적용하려고 할 때, 이 변경사항들이 새로운 자식 프로세스에 자동으로 반영되지 않기 때문에 문제가 발생할 수 있음. 이는 자식 프로세스가 부모 프로세스의 현재 상태를 “복제”하지 않기 때문.

Spawn vs fork

fork

- fork는 유닉스(Unix) 및 유닉스 기반 운영 체제에서 사용되는 시스템 콜(System call) 중 하나.
- C, C++, 자바 등의 언어에서 주로 사용.
- 부모 프로세스의 상태와 메모리를 복제하여 자식 프로세스를 생성.
- 상대적으로 빠르게 프로세스를 생성할 수 있음.
- 부모 프로세스와 자식 프로세스는 메모리를 공유.
 - **현재 프로세스의 모든 state (생성한 객체, 열려있는 파일, import된 것들까지) 전부 메모리에서 그대로 복사 후 타겟 함수 실행**
- 병렬 처리를 수행할 때 자주 사용됨.

● 장점:

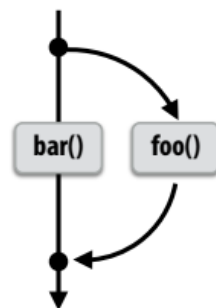
- 성능: 부모 프로세스의 메모리 페이지를 복제하기 때문에, 새 프로세스 생성이 매우 빠름. 특히 Linux 같은 시스템에서 효율적임.
- 자원 효율성: 초기 자원 사용량이 적고, 프로세스 생성 시간이 짧습니다. copy-on-write 기술 덕분에 메모리 페이지는 실제로 수정될 때까지 부모와 자식 프로세스 간에 공유될 수 있음.

● 단점:

- **이식성 문제:** Windows와 같은 일부 운영 체제에서는 지원되지 않음.
- **안정성 문제:** 부모 프로세스의 자원과 상태를 그대로 복제하기 때문에, 파일 핸들러나 소켓과 같은 자원을 공유하게 되어, 예상치 못한 상태 공유 문제가 발생할 수 있음.
- **메모리 누수:** 부모 프로세스가 큰 메모리를 사용하고 있을 경우, 자식 프로세스도 동일한 양의 메모리를 사용할 것으로 예상되어, 시스템의 메모리 사용량이 크게 증가할 수 있음.

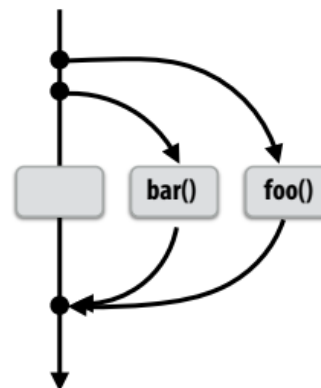
Basic Cilk Plus examples

```
// foo() and bar() may run in parallel
cilk_spawn foo();
bar();
cilk_sync;
```

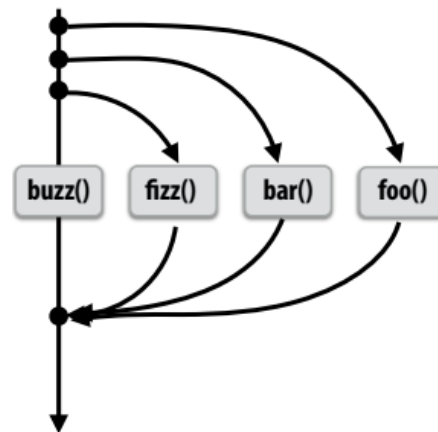


```
// foo() and bar() may run in parallel
cilk_spawn foo();
cilk_spawn bar();
cilk_sync;
```

Same amount of independent work first example, but potentially higher runtime overhead (due to two spawns vs. one)

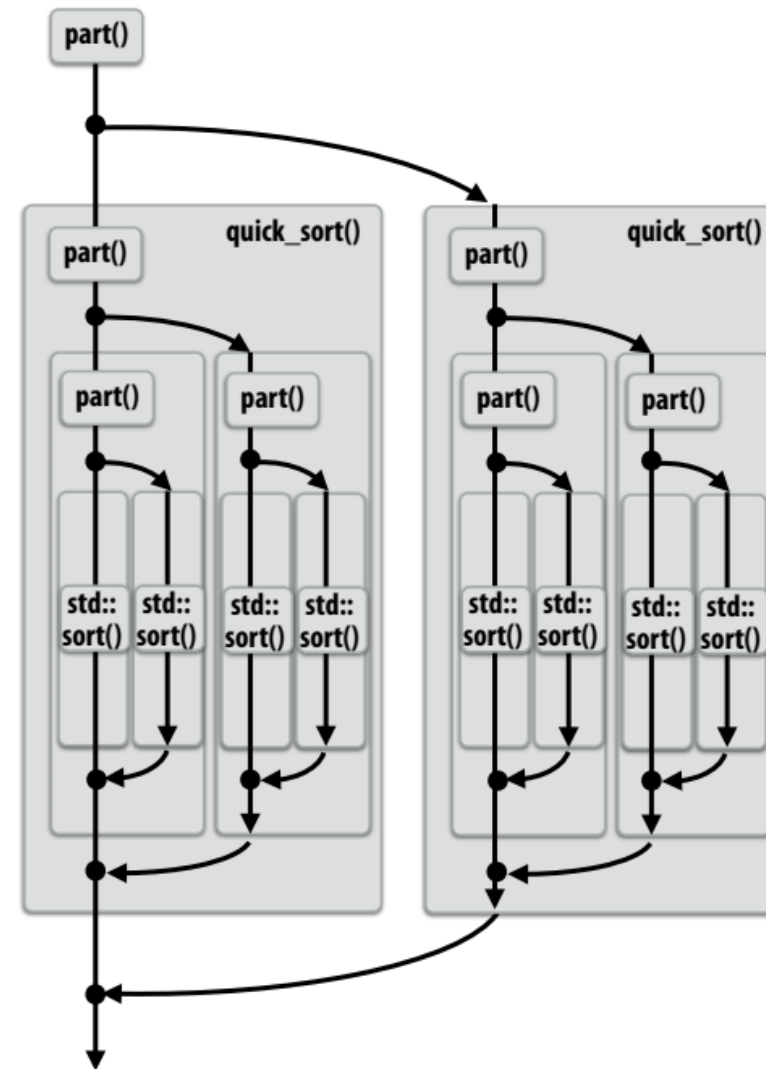


```
// foo, bar, fizz, buzz, may run in parallel
cilk_spawn foo();
cilk_spawn bar();
cilk_spawn fizz();
buzz();
cilk_sync;
```



```
void quick_sort(int* begin, int* end) {  
    if (begin >= end - PARALLEL_CUTOFF)  
        std::sort(begin, end);  
    else {  
        int* middle = partition(begin, end);  
        cilk_spawn quick_sort(begin, middle);  
        quick_sort(middle+1, last);  
    }  
}
```

문제 크기가 충분히 작은 경우 순차적으로 정렬(sort)함
(스폰의 오버헤드가 잠재적 병렬화의 이점을 능가함).

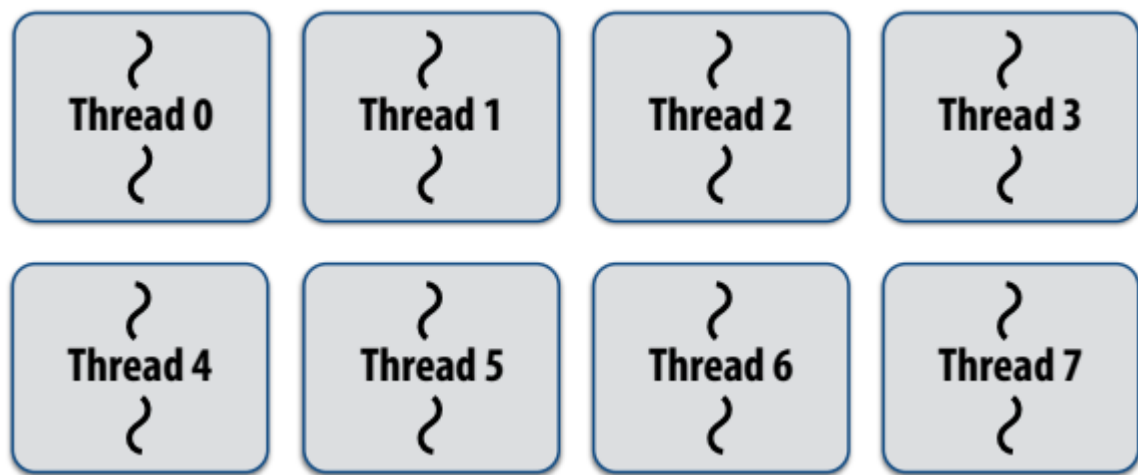


- 주요 아이디어: `cilk_spawn`을 사용하여 독립적인 작업(잠재적 병렬 처리)을 시스템에 노출.
- 병렬 프로그래밍 경험 법칙 기억하기
 - 병렬 실행 역량(parallel execution capability)만큼의 작업을 원함(예: 프로그램이 컴퓨터의 모든 처리 리소스(machine's processing resources)를 채우는 데 필요한 만큼의 작업을 생성해야 함).
 - 코어에 대한 모든 작업의 워크로드 균형이 잘 맞도록 실행 기능보다 더 많은 독립 작업을 원함.
 - “병렬 여유” = 머신의 병렬 실행 능력에 대한 독립 작업의 비율(실제로는 ~8이 좋은 비율임)
 - - 그러나 독립 작업이 너무 많아서 작업의 세분성이 너무 작아서는 안 됨(너무 많은 슬랙은 세분화된 작업(fine-grained work)을 관리하는 데 오버헤드가 발생함).

- 매우 간단한 스케줄러를 고려:
 - `pthread_create`를 사용하여 각 `cilk_spawn`에 대해 `pthread`를 실행.
 - `cilk_sync`를 적절한 `pthread_join` 호출로 변환.
- 잠재적인 성능 문제?
 - 헤비급 스폰 작업
 - 코어보다 더 많은 스레드가 동시에 실행됨
 - 컨텍스트 전환 오버헤드
 - 필요 이상으로 큰 작업 세트, 적은 캐시 로컬리티

참고: 이제 cilk의 구현에 대해 이야기하겠습니다.

- Cilk Plus 런타임은 워커 스레드 풀을 유지.
 - 생각해 보세요: 모든 스레드는 애플리케이션 실행 시 생성*.
 - 머신의 실행 컨텍스트 수만큼 정확히 많은 워커 스레드가 생성.



예시: 하이퍼스레딩을 사용하는 쿼드코어 노트북을 위한 8개의 스레드 워커 풀

```
while (work_exists()) {  
    work = get_new_work();  
    work.run();  
}
```

* 이런 식으로 생각하면 완벽하겠지만, 실제로 런타임은 게으르고 첫 번째 Cilk spawn에서 worker threads를 초기화하는 경향이 있음. (이것은 일반적인 구현 전략. ISPC는 ISPC 작업을 실행하는 워커 스레드에서도 동일하게 수행.)

다음 코드를 실행해 보자

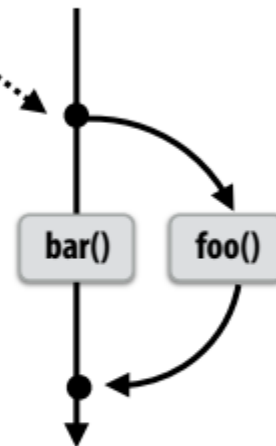
<https://cilplus.github.io/>

구체적으로, `foo()`가 생성된 시점부터 실행을 고려합니다.

```
cilk_spawn foo();  
bar();  
cilk_sync;
```

spawned child

continuation (rest of calling function)



Assignment question: `foo()`와 `bar()`는 어떤 스레드에서 실행되어야 하나요?

}
Thread 0
{

}
Thread 1
{

Cilk Plus는 분할 정복(Divide-and-Conquer) 알고리즘과 같은 작업에서 병렬성을 쉽게 표현하도록 설계

Cilk Plus의 런타임 시스템은 이러한 병렬 실행을 효율적으로 관리하기 위해 Work-Stealing 스케줄러를 사용.

- **워커 스레드 풀:** Cilk 런타임은 일반적으로 시스템의 CPU 코어(또는 하이퍼스레딩 포함 실행 컨텍스트) 수만큼 워커 스레드 풀을 유지함.
- **Spawn 처리 (Child-First/Continuation Stealing):** 스레드가 `cilk_spawn foo();`를 만나면, 대부분의 Cilk 구현 (문서에서 설명하는 방식)은 "Child-First" 전략을 사용.
 - 현재 스레드(예: 스레드 0)는 즉시 `foo()` 함수(spawned child)를 실행하기 시작함.
 - `cilk_spawn` 다음의 코드, 즉 `bar();`를 포함하는 나머지 작업(continuation)은 스레드 0의 로컬 작업 큐(work queue, 보통 deque로 구현)에 넣어둠.
- **Work Stealing:** 만약 다른 워커 스레드(예: 스레드 1)가 현재 할 일이 없어 유휴 상태(idle)가 되면, 다른 스레드(여기서는 스레드 0)의 작업 큐를 살펴봄.
 - 스레드 1은 스레드 0의 큐에서 작업을 "훔쳐"(steal) 올 수 있음. 이때 일반적으로 큐의 앞부분(head)에서 가장 오래된(큰) 작업을 훔쳐감. 이 예에서는 `bar()`를 포함하는 continuation을 훔쳐갈 수 있음.
 - 스레드 1이 `bar()`를 실행하게 되면, 스레드 0에서 실행 중인 `foo()`와 실제로 병렬 실행이 이루어짐.
- **Sync 처리:** 스레드가 `cilk_sync`에 도달했을 때, 만약 해당 스레드가 이전에 spawn 했지만 다른 스레드에 의해 훔쳐진(stolen) 작업이 아직 완료되지 않았다면, 해당 작업들이 모두 완료될 때까지 기다림. 만약 아무 작업도 훔쳐지지 않았고 spawn된 자식 함수(`foo()`)의 실행이 이미 끝났다면, `cilk_sync`는 아무 작업도 하지 않고 바로 통과(no-op).

Cilk Plus는 분할 정복(Divide-and-Conquer) 알고리즘과 같은 작업에서 병렬성을 쉽게 표현하도록 설계

- `cilk_spawn`을 통해 `foo()` 함수를 병렬 실행 가능하도록 만들고, 현재 스레드는 `bar()` 함수를 계속 실행함.
- `cilk_sync`는 `foo()` 함수의 완료를 보장하는 지점이다.
- 이 과정은 cilk 런타임의 Work-Stealing 스케줄러에 의해 효율적으로 관리되어 유휴 스레드가 다른 스레드의 작업을 훔쳐와 처리함으로써 실제 병렬 실행을 가능하게 함. 이는 부하 균형(load balancing)과 지역성(locality)을 고려한 효율적인 스케줄링 방식

보통 순차 구현에서는 일반 함수 호출을 통해 자식 먼저 실행...

- 스레드가 `foo()`를 실행한 다음, `foo()`에서 반환한 다음, `bar()`를 실행.
- 스레드의 스택에 연속이 암시되어 있음.



Executing `foo()`...

기존 스레드 호출 스택
(`foo()`에서 반환된 후 다음에 `bar()`
가 실행됨을 나타냄)

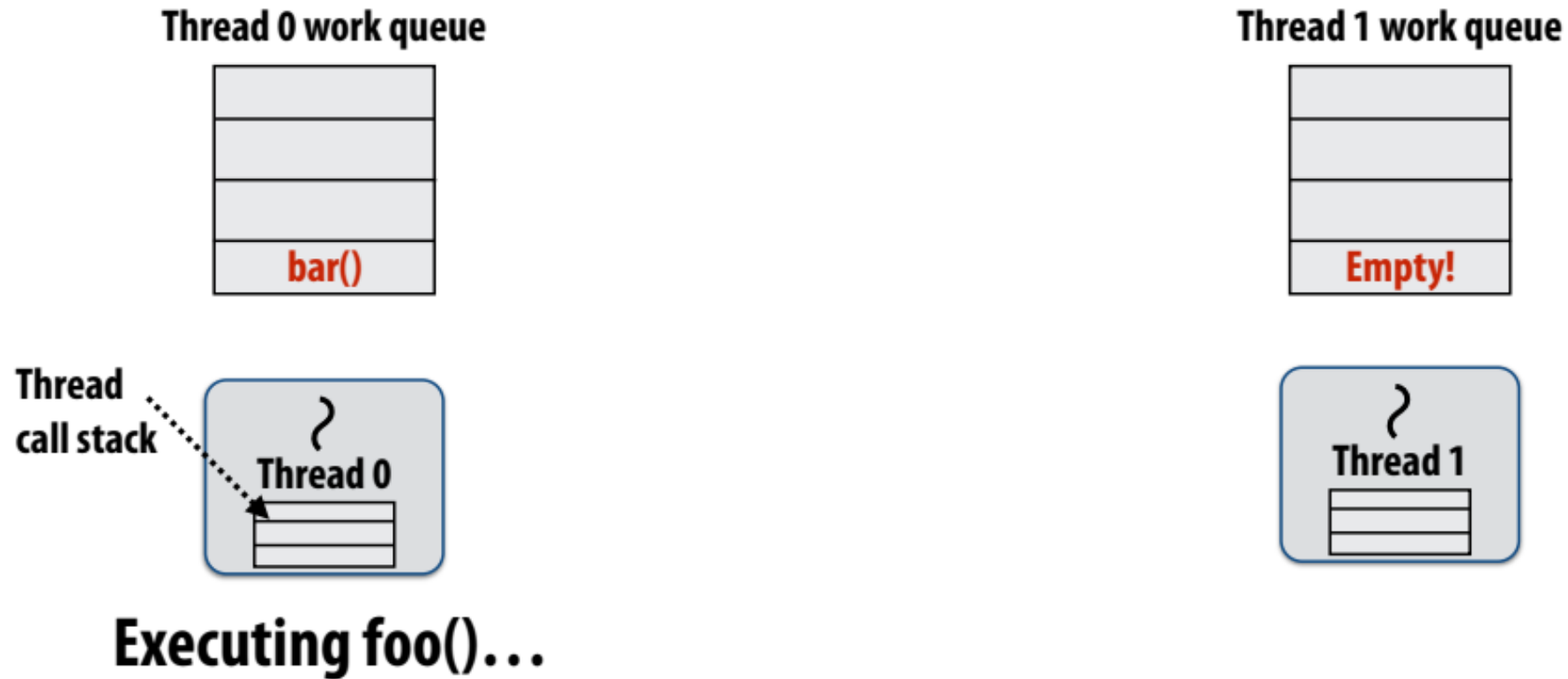


`foo()`를 실행하는 동안 스레드 1이 유휴 상태가 되면 어떻게 될까요...?

이때 스레드 1이 `bar()`를 실행하고 있을 수 있습니다!

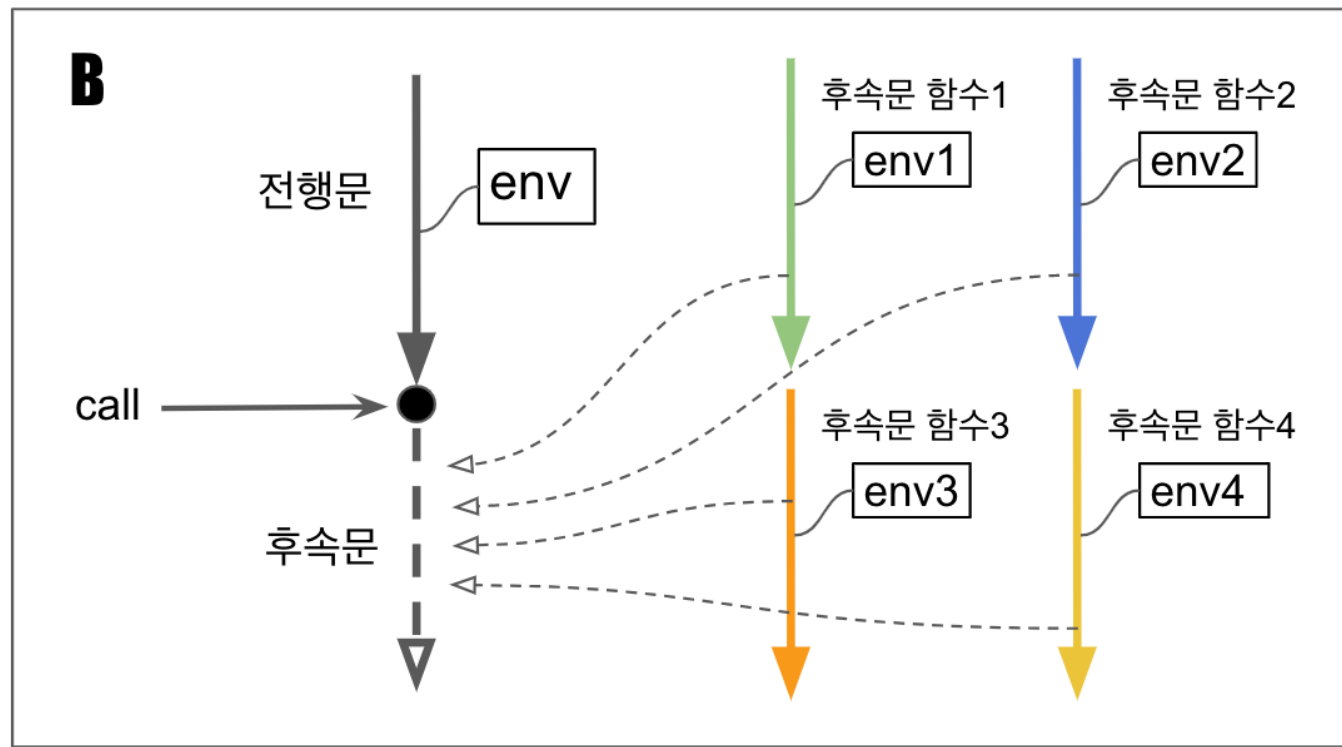
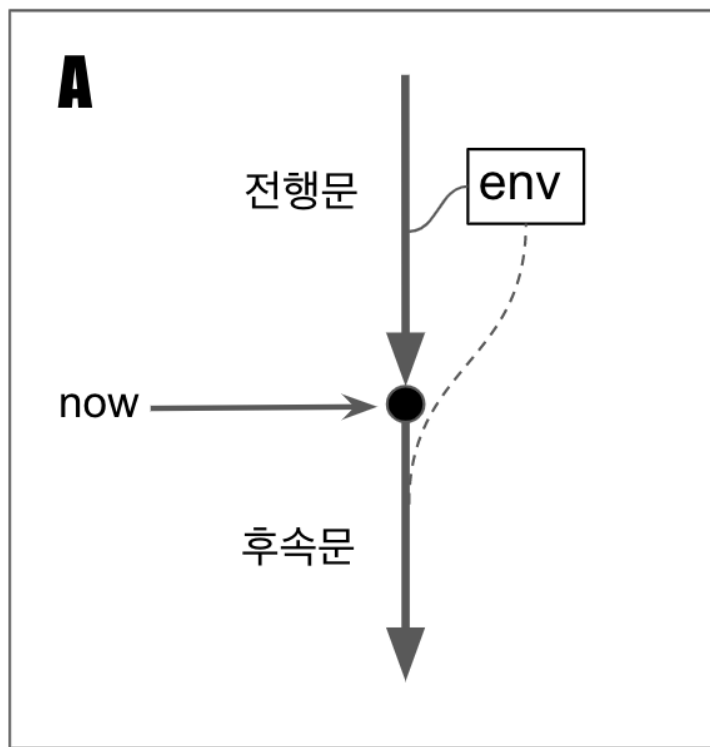
스레드별 작업 Queues는 “할 작업(work to do)”을 저장

- `cilk_spawn foo()`에 도달하면 스레드는 작업 대기열(work queue)에 후속문(Continuation)을 배치하고 `foo()` 실행을 시작.



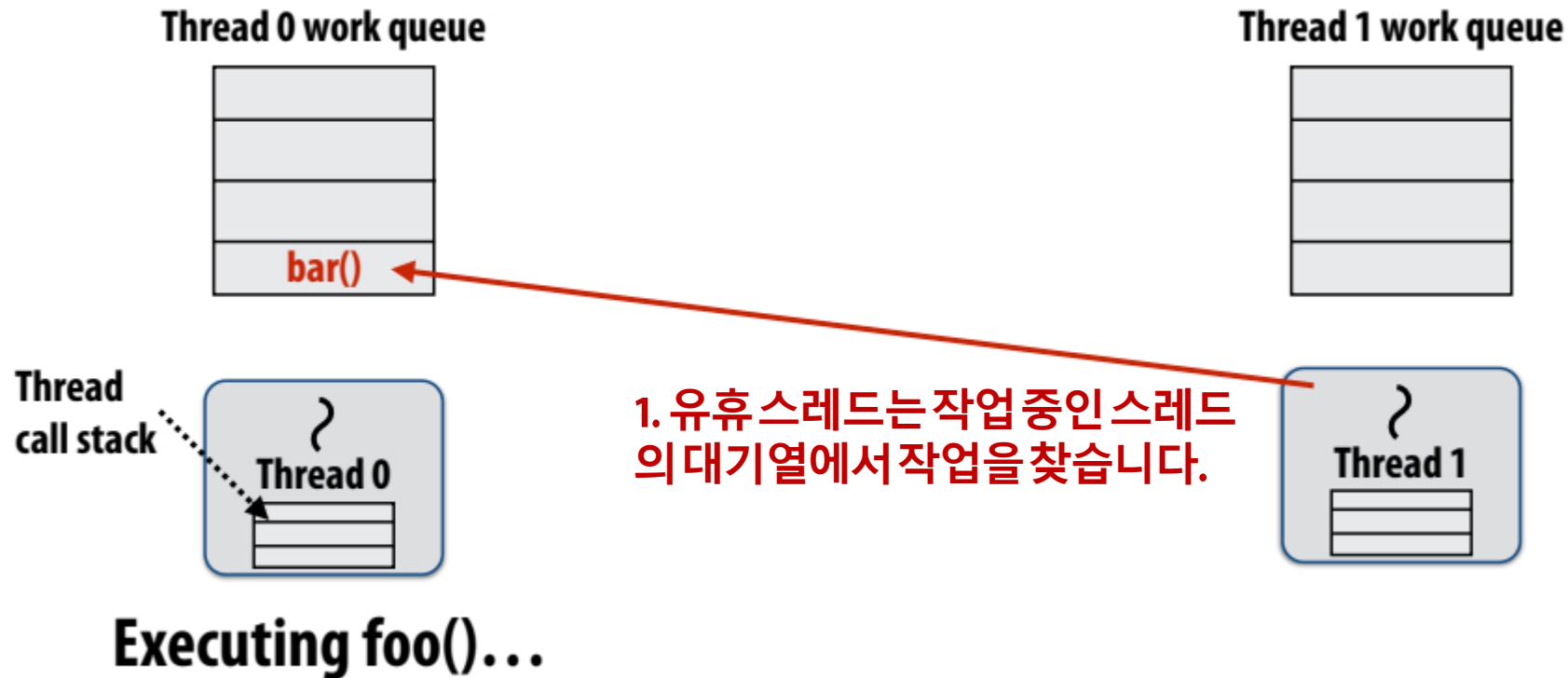
후속문(Continuation)

- 후속문(Continuation)은 프로그램 제어에 관한 추상화 개념이다. 모든 프로그램은 후속문 관점으로 볼 수 있다.
- if 문은 2가지 후속문 중 선택이며, 예외(Exception)는 후속문을 버리는 것이고, for 문은 후속문의 반복이다.
- 모든 언어는 후속문을 암묵적으로 사용하고 있지만, 후속문을 일급 객체(First Class)로서 명시적으로 지원하는 언어들은 Lisp나 ML 계열의 일부 비주류 언어들뿐이었다.
- 하지만 최근에는 Python([Stackless Python](#)), Ruby([Continuation](#))뿐만 아니라, C++([boost.context.continuation](#))이나 Java([Loom Project](#)), Javascript([rhino](#)) 등 주류 언어들에서도 후속문을 지원하거나 지원 예정에 있다.



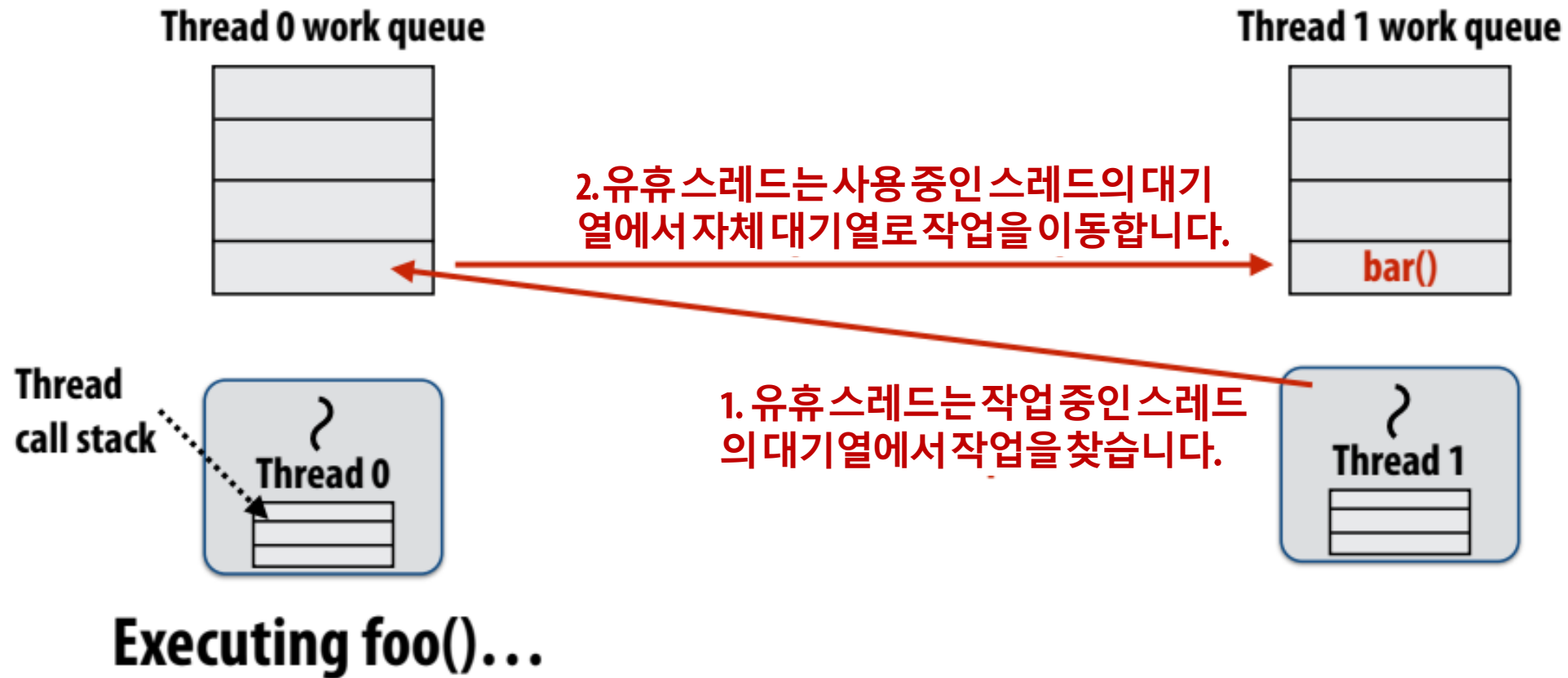
유틸 스레드가 사용 중인 스레드에서 작업을 “훔치기(steal)”

스레드 1이 유틸 상태가 되면(즉, 자체 대기열(queue)에 작업이 없는 경우) 스레드 0의 대기열에서 일을 찾을.



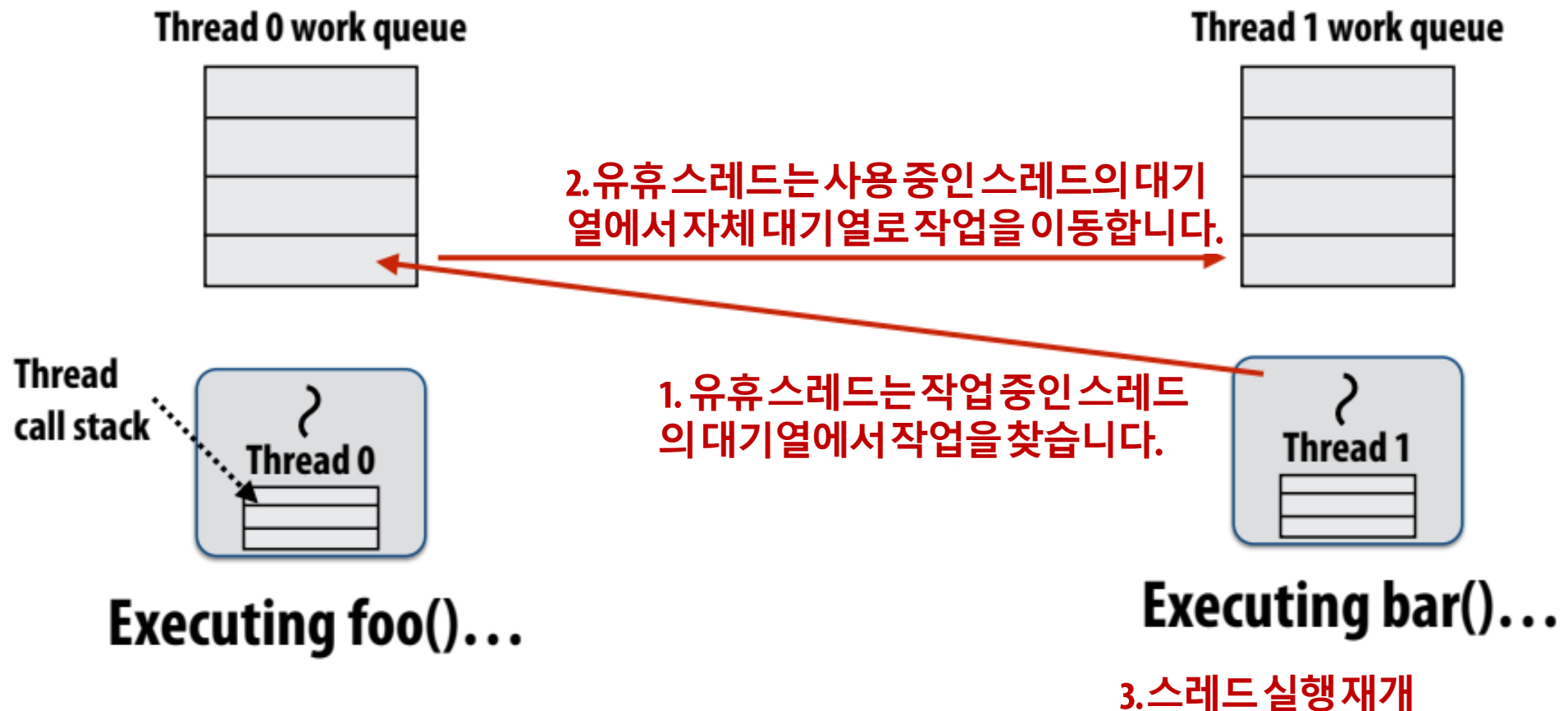
유틸 스레드가 사용 중인 스레드에서 작업을 “훔치기(steal)”

스레드 1이 유틸 상태가 되면(즉, 자체 대기열(queue)에 작업이 없는 경우) 스레드 0의 대기열에서 일을 찾음.



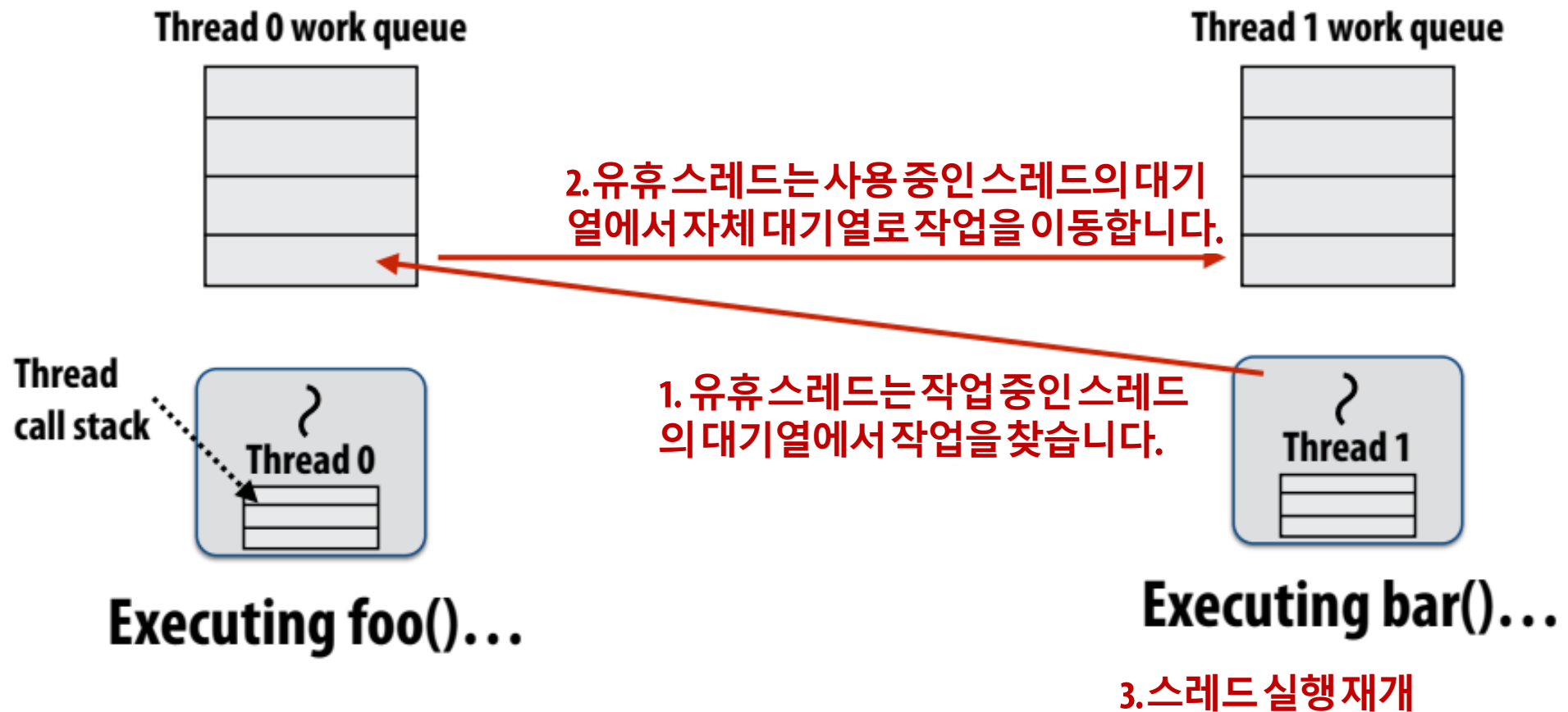
유틸 스레드가 사용 중인 스레드에서 작업을 “훔치기(steal)”

스레드 1이 유틸 상태가 되면(즉, 자체 대기열(queue)에 작업이 없는 경우) 스레드 0의 대기열에서 일을 찾음.



유틸 스레드가 사용 중인 스레드에서 작업을 “훔치기(steal)”

스레드 1이 유틸 상태가 되면(즉, 자체 대기열(queue)에 작업이 없는 경우) 스레드 0의 대기열에서 일을 찾음.

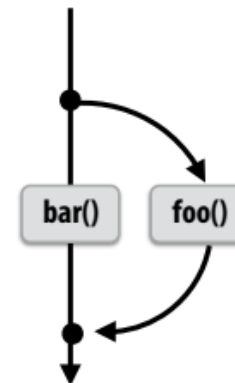


스폰 (spawn)시 호출 스레드가 자식을 실행할까요, 아니면 continuation을 실행할까요?

```
cilk_spawn foo();  
bar();  
cilk_sync;
```

spawned child

continuation (rest of calling function)
나머지 호출 함수

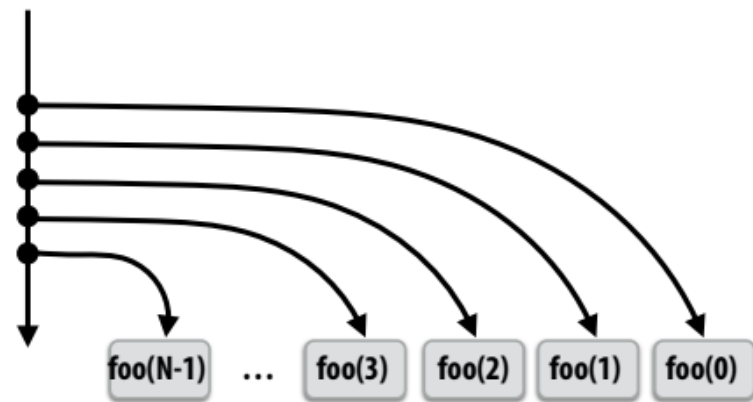


- Continuation(후속문) 실행을 먼저 실행: 나중에 실행할 자식을 대기열에 대기시킴.
 - 다른 스레드에서 자식을 훔칠 수 있도록 설정("자식 훔치기(child stealing)")
- 자식(child) 먼저 실행: 나중에 실행하기 위해 Continuation을 큐에 넣음.
 - 다른 스레드가 연속을 훔칠 수 있도록 함("후속문 훔치기(continuation stealing)").
- 어떤 구현을 선택할까요?

Consider thread executing the following code

스레드 1이 유휴 상태가 되면(즉, 자체 대기열(queue)에 작업이 없는 경우) 스레드 0의 대기열에서 일을 찾을 수 있다.

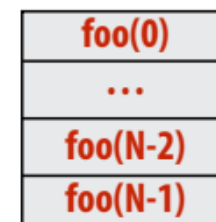
```
for (int i=0; i<N; i++) {  
    cilk_spawn foo(i);  
}  
cilk_sync;
```



● continuation을 먼저 실행("자식 훔치기(child stealing)")

- 호출자 스레드 스폰은 하나라도 반복을 실행하기 전에 모든 반복에 대해 작동.
- 호출 그래프의 넓비 우선 순위를 생각해 보자. 스폰된 작업을 위한 $O(N)$ 공간(최대 공간)
- 스틸링이 없으면 실행 순서는 `cilk_spawn`이 제거된 프로그램과 매우 다름.

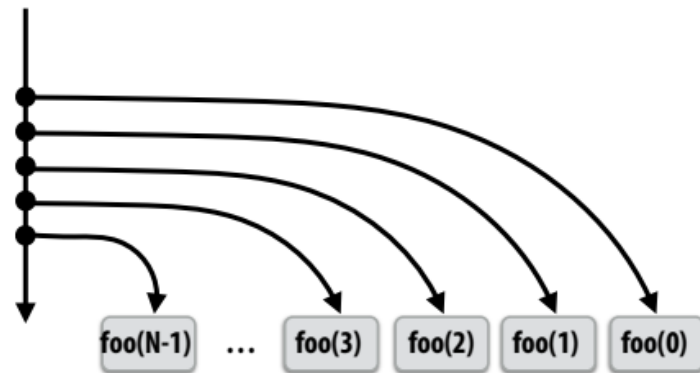
Thread 0 work queue



Consider thread executing the following code

스레드 1이 유힬 상태가 되면(즉, 자체 대기열(queue)에 작업이 없는 경우) 스레드 0의 대기열에서 일을 찾음.

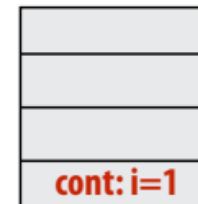
```
for (int i=0; i<N; i++) {  
    cilk_spawn foo(i);  
}  
cilk_sync;
```



● 자식 먼저 실행("continuation 훔치기")

- 호출자 스레드는 훔칠 항목을 하나만 생성 (나머지 모든 반복을 나타내는 continuation).
- 훔칠 항목이 없으면 스레드는 작업 큐에서 continuation 을 계속 팝하고 새 continuation 을 큐에 넣음(업데이트된 값 i로).
- 실행 순서는 스폰이 제거된 프로그램과 동일함.
- Think: 호출 그래프의 깊이 우선 탐색

Thread 0 work queue

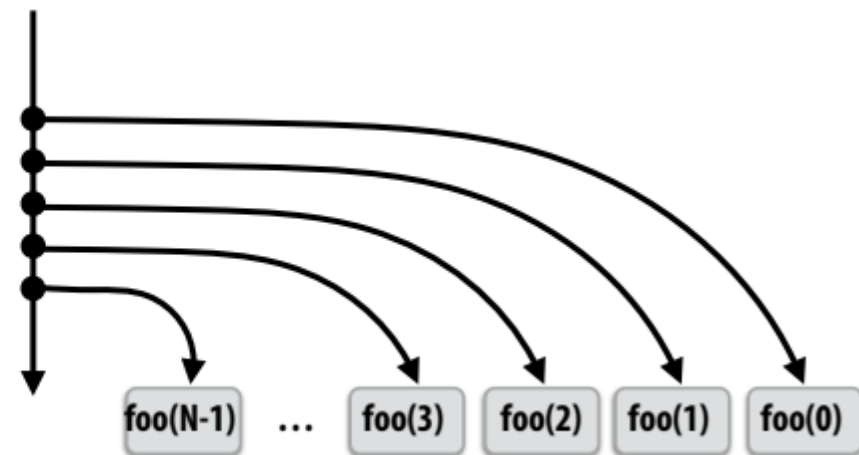


Executing foo(0)...

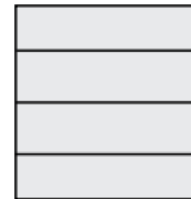
Consider thread executing the following code

스레드 1이 유티 상태가 되면(즉, 자체 대기열(queue)에 작업이 없는 경우) 스레드 0의 대기열에서 일을 찾음.

```
for (int i=0; i<N; i++) {  
    cilk_spawn foo(i);  
}  
cilk_sync;
```

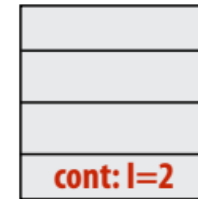


Thread 0 work queue



Executing foo(0)...

Thread 1 work queue



Executing foo(1)...

● 자식 먼저 실행("continuation 훔치기")

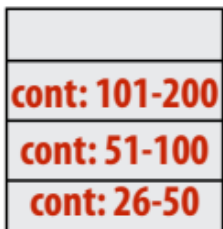
- `i`가 1만큼 진행된 Continuation을 큐에 넣음.
- Continuation을 훔치면, 훔친 스레드가 다음 반복을 생성하고 실행.
- T 스레드가 있는 시스템의 작업 큐 스토리지가 단일 스레드 실행을 위한 스택 스토리지의 T 배를 넘지 않음을 증명할 수 있음.

Scheduling quicksort: assume 200 elements

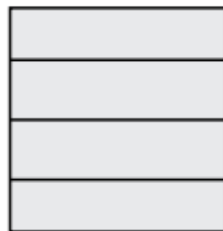
```
void quick_sort(int* begin, int* end) {  
    if (begin >= end - PARALLEL_CUTOFF)  
        std::sort(begin, end);  
    else {  
        int* middle = partition(begin, end);  
        cilk_spawn quick_sort(begin, middle);  
        quick_sort(middle+1, last);  
    }  
}
```

다른 스레드가 **queue** 에서 어떤 작업을 훑쳐야 하나요?
(예: 위쪽 또는 아래쪽에서 훑치기)

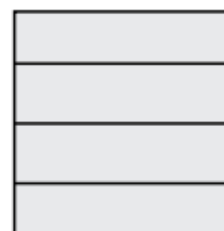
Thread 0 work queue



Thread 1 work queue



Thread 2 work queue



...

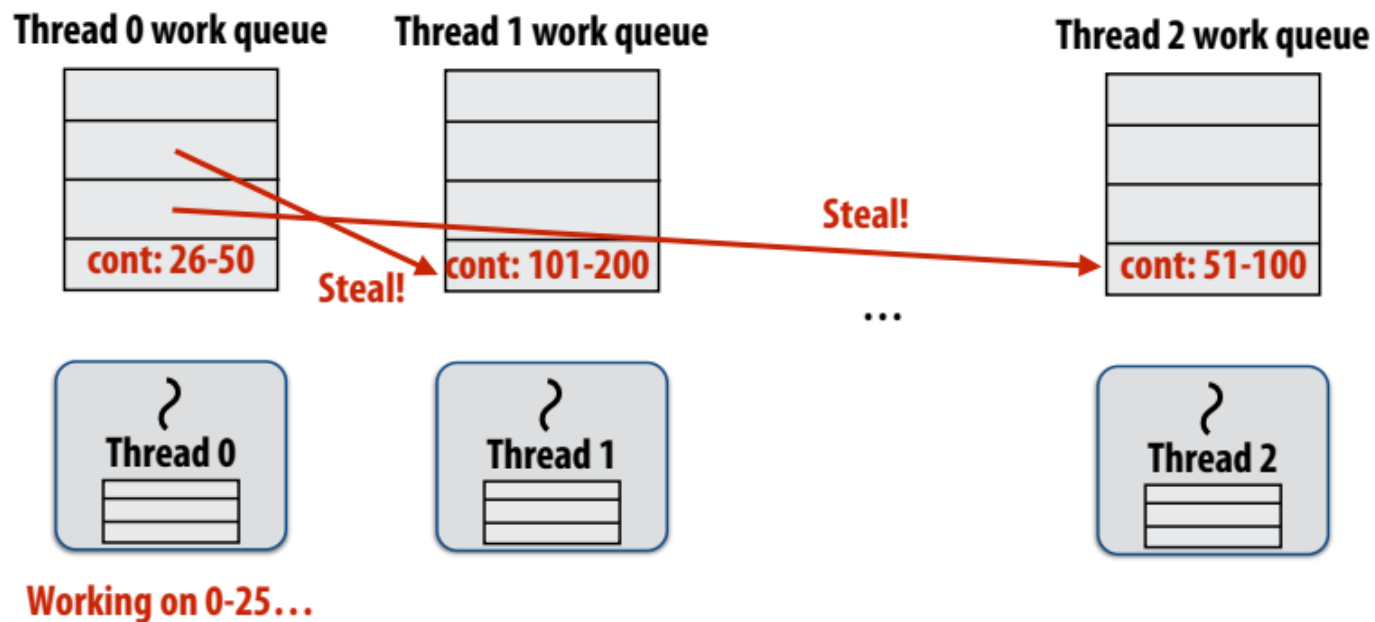


Working on 0-25...

Implementing work stealing: dequeue per worker

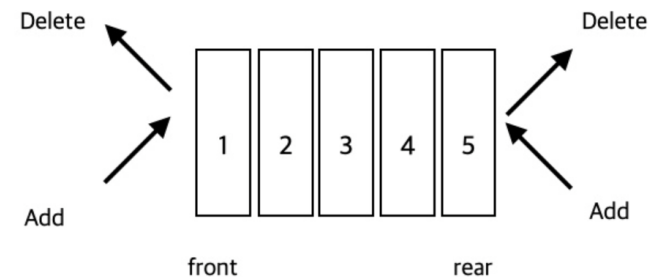
dequeue로 구현된 Work queue (double ended queue)

- 로컬 스레드는 “tail”(bottom)에서 푸시/팝업함.
- 원격 스레드는 “head”(top)에서 훔침.



Deque이란?

- Dequeue란 double-ended queue의 약자입니다.
- 스택과 큐를 합친 자료구조로서 양쪽 모두에서 삽입과 삭제가 가능합니다.



Implementing work stealing: dequeue per worker

dequeue로 구현된 Work queue (double ended queue)

- 로컬 스레드는 "tail"(bottom)에서 푸시/팝업함.
- 원격 스레드는 "head"(top)에서 훔침.

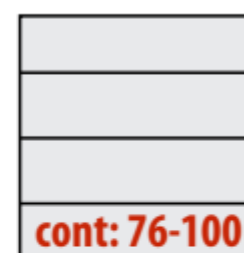
Thread 0 work queue



Thread 1 work queue



Thread 2 work queue



...



Working on 0-25...



Working on 101-150...



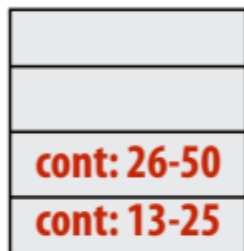
Working on 51-75...

Implementing work stealing: dequeue per worker

dequeue로 구현된 Work queue (double ended queue)

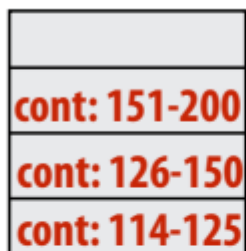
- 로컬 스레드는 "tail"(bottom)에서 푸시/팝업함.
- 원격 스레드는 "head"(top)에서 훔침.

Thread 0 work queue



Working on 0-12...

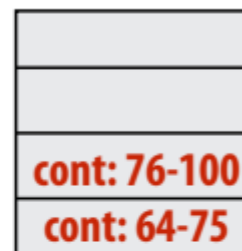
Thread 1 work queue



Working on 101-113...

...

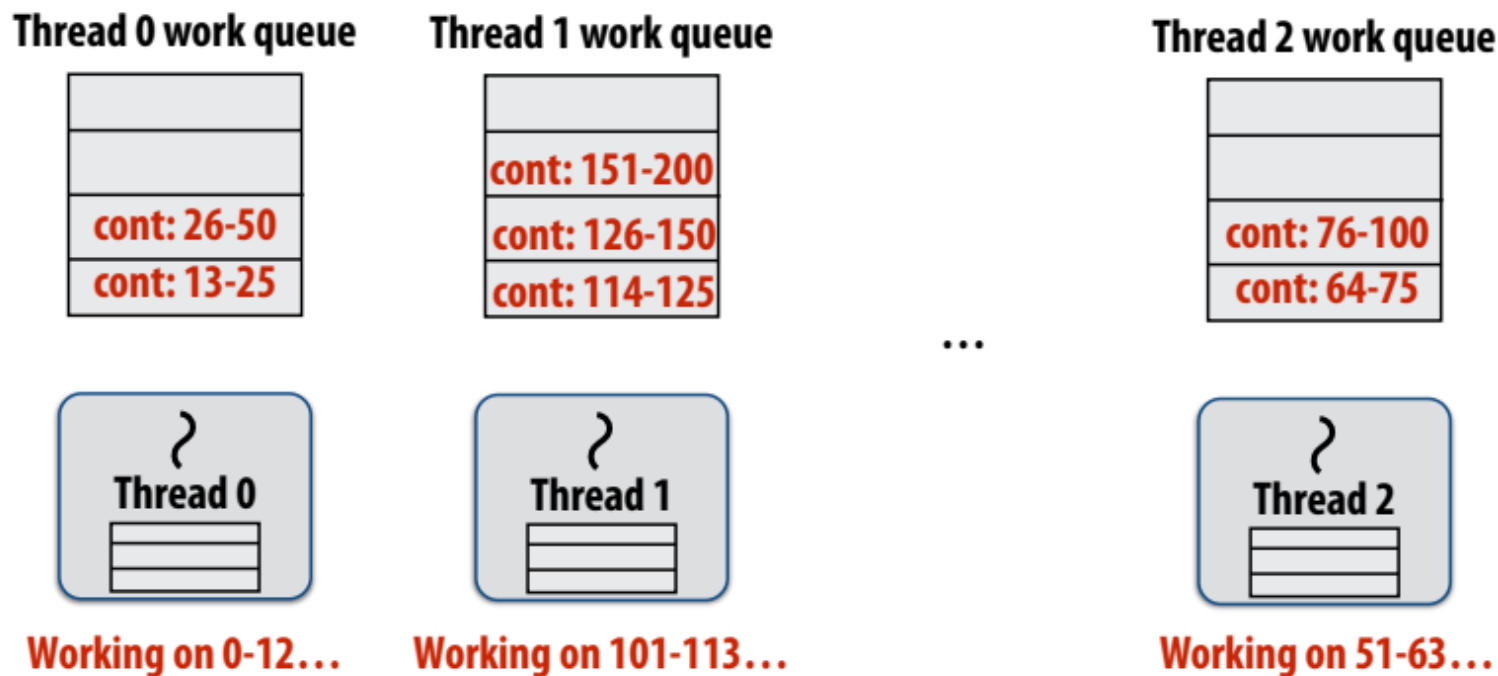
Thread 2 work queue



Working on 51-63...

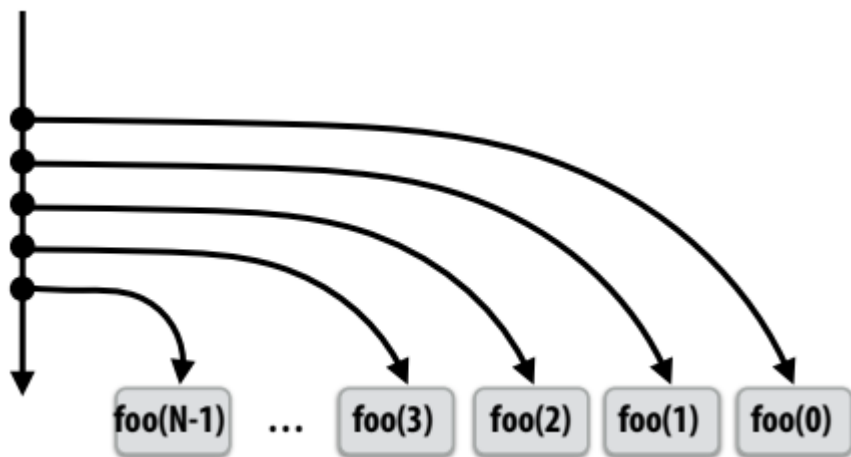
Implementing work stealing: choice of victim(작업 훔치기 구현: 희생자 선택)

- 유휴 스레드가 무작위로 스레드를 선택하여 도용(steal)을 시도함.
- **dequeue** 맨 위에서 작업 훔치기:
 - 가장 많은 양의 작업 훔치기(훔치는 횟수 감소)
 - 각 스레드가 수행하는 작업의 최대 로컬리티(자식 우선 실행 방식과 결합 시)
 - 훔치는 스레드와 로컬 스레드가 동일한 **dequeue** 요소에 대해 경합하지 않음
(효율적인 잠금 없는 dequeue 구현이 존재함)

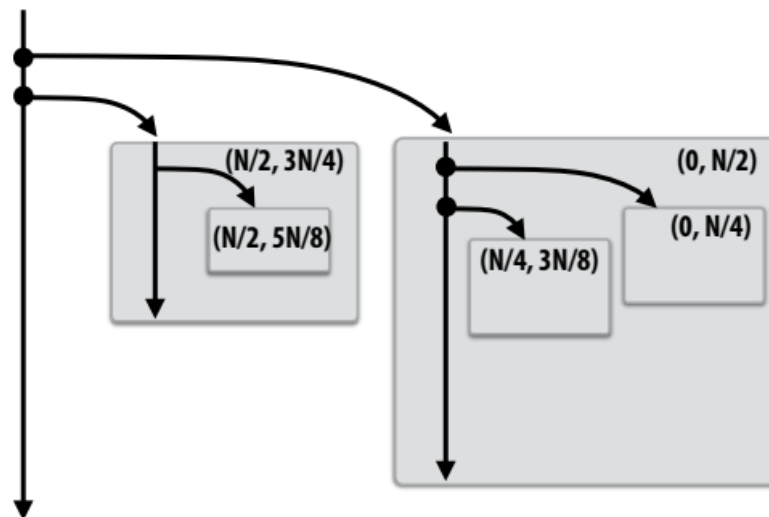


Child-first work stealing scheduler anticipates divide-and-conquer parallelism

```
for (int i=0; i<N; i++) {  
    cilk_spawn foo(i);  
}  
cilk_sync;
```



```
void recursive_for(int start, int end) {  
    while (start <= end - GRANULARITY) {  
        int mid = (end - start) / 2;  
        cilk_spawn recursive_for(start, mid);  
        start = mid;  
    }  
    for (int i=start; i<end; i++)  
        foo(i);  
}  
recursive_for(0, N);
```



- 오른쪽의 코드는 병렬로 작업을 생성하므로(왼쪽의 코드는 그렇지 않음) 병렬 머신을 더 빨리 채웁니다.

Child-first work stealing scheduler anticipates divide-and-conquer parallelism

<https://velog.io/@vies00/Java-work-stealing-fork-join-rljtjnfly>

<https://blog.naver.com/jjoommnn/130044743051>

<https://howudong.tistory.com/88>

ワークスティール

<https://zenn.dev/nextzlog/books/parallel-work-stealing-scheduler-on-d/viewer/1>

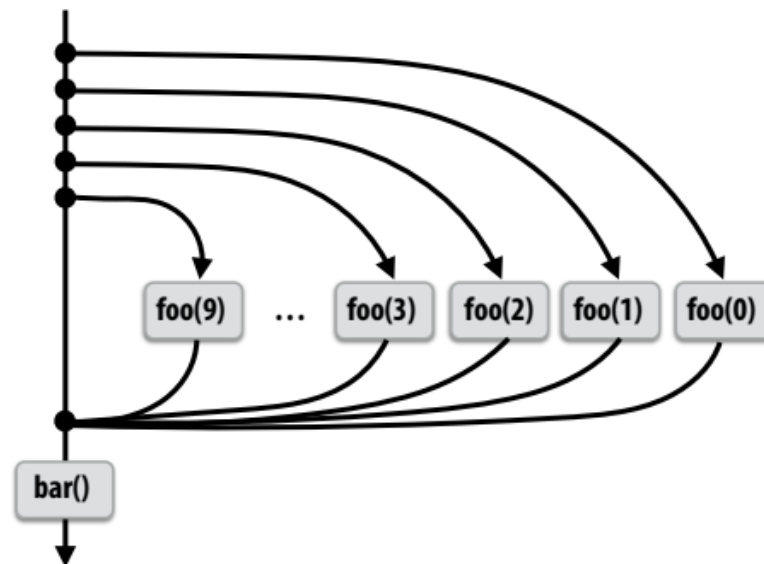
<https://www.slideshare.net/slideshow/ss-13914526/13914526>

<https://postd.cc/scheduler/>

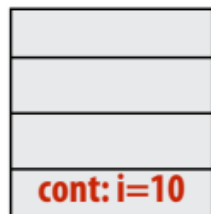
<https://www.slideshare.net/slideshow/fork-join/16914627#4>

Implementing sync

```
for (int i=0; i<10; i++) {  
    cilk_spawn foo(i);  
}  
cilk_sync;  
bar();
```

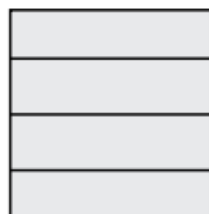


Thread 0 work queue



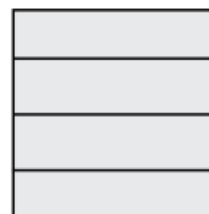
Working on foo(9)...

Thread 1 work queue



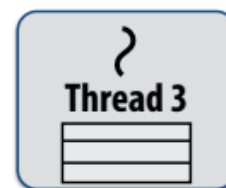
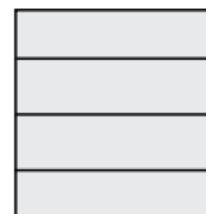
Working on foo(7)...

Thread 2 work queue



Working on foo(8)...

Thread 3 work queue



Working on foo(6)...

loop에서 모든 작업이
거의 완료되었을 때의
워커 스레드 상태

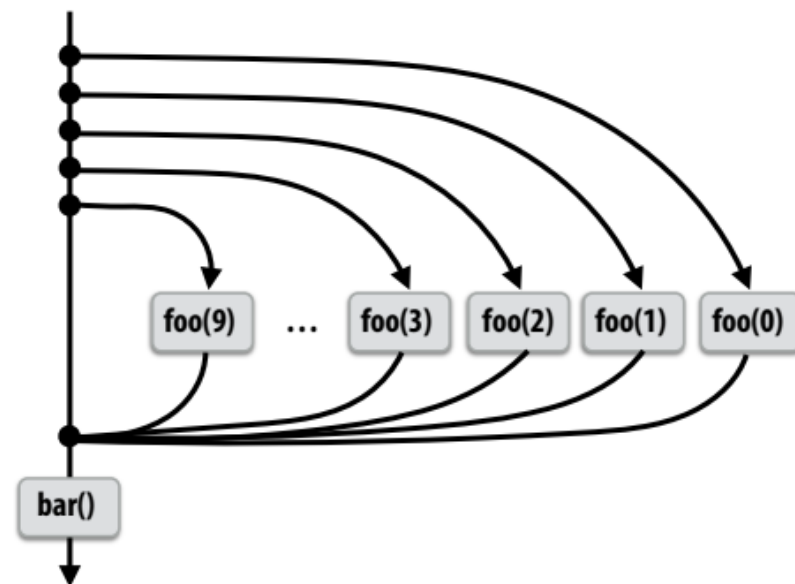


Implementing sync: no stealing case

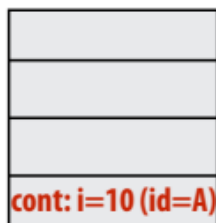
block (id: A)

```
for (int i=0; i<10; i++) {  
    cilk_spawn foo(i);  
}
```

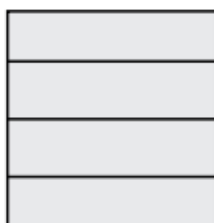
cilk_sync; 블록 A 내에서 spawn된 모든 호출에 대한 동기화
bar();



Thread 0 work queue



Thread 1 work queue



Working on foo(9), id=A...

다른 스레드에 의해 작업이 도난당하지 않았다면
동기화 지점에서 할 일이 없음.

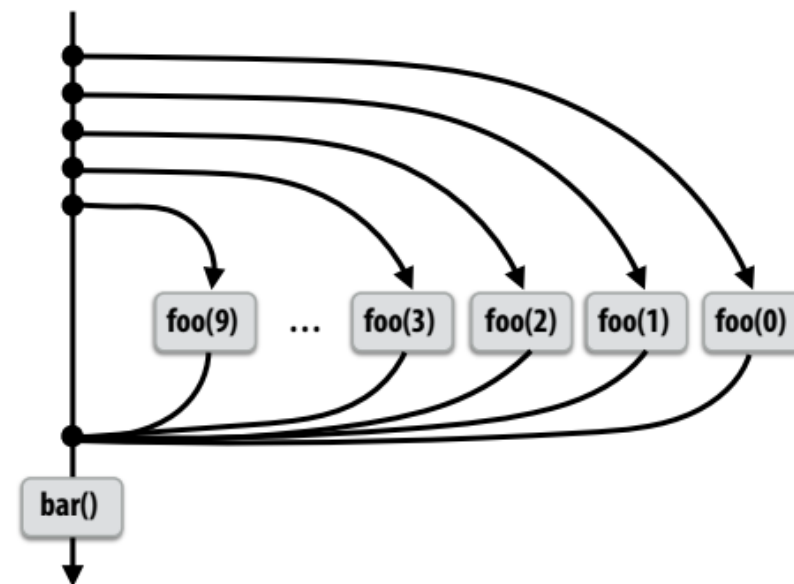
cilk_sync is a no-op.

Implementing sync: stealing case

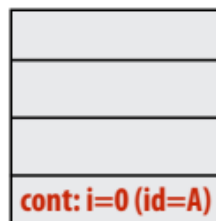
block (id: A)

```
for (int i=0; i<10; i++) {  
    cilk_spawn foo(i);  
}
```

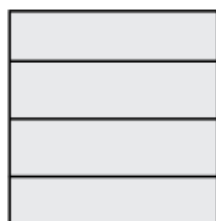
cilk_sync; 블록 A 내에서 spawn된 모든 호출에 대한 동기화
bar();



Thread 0 work queue



Thread 1 work queue



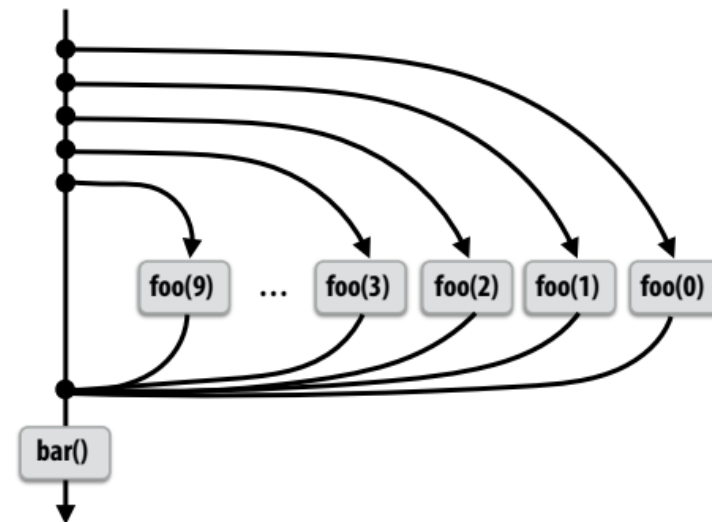
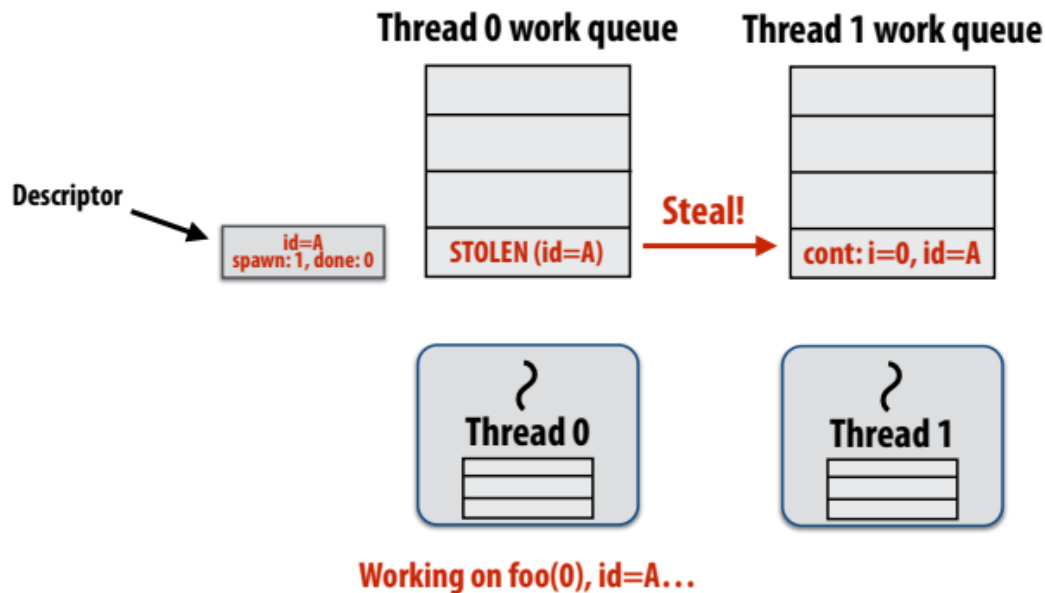
Working on foo(0), id=A...

Implementing sync: stealing case

block (id: A)

```
for (int i=0; i<10; i++) {  
    cilk_spawn foo(i);  
}
```

cilk_sync; 블록 A 내에서 spawn된 모든 호출에 대한 동기화
bar();



idle 스레드 1이 busy 스레드 0에서 훔침
참고: 블록 A에 대한 descriptor 생성

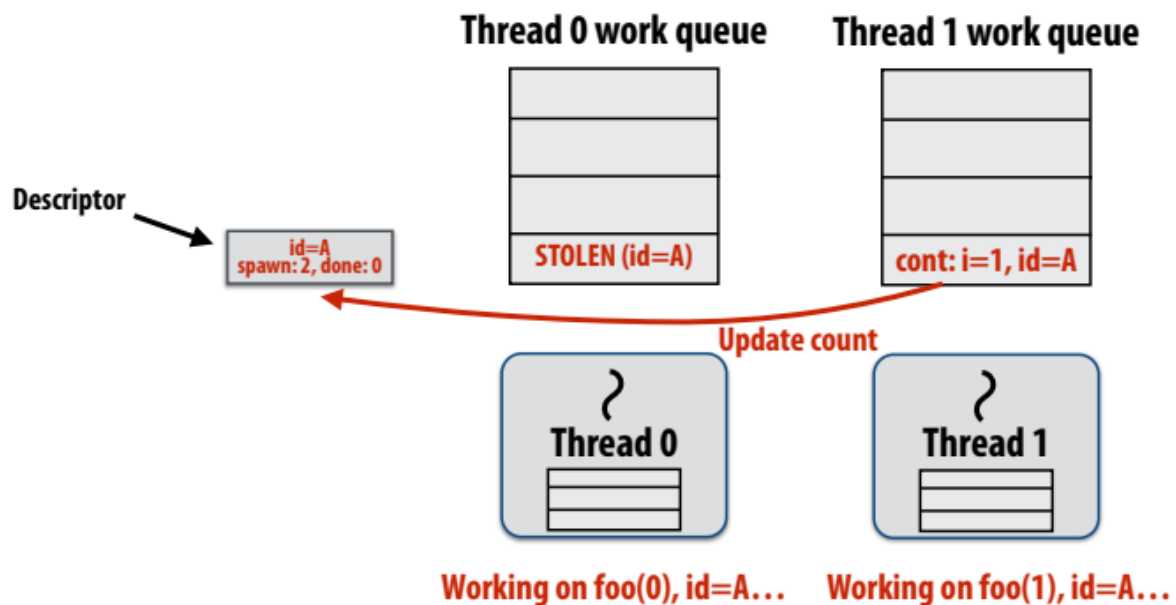
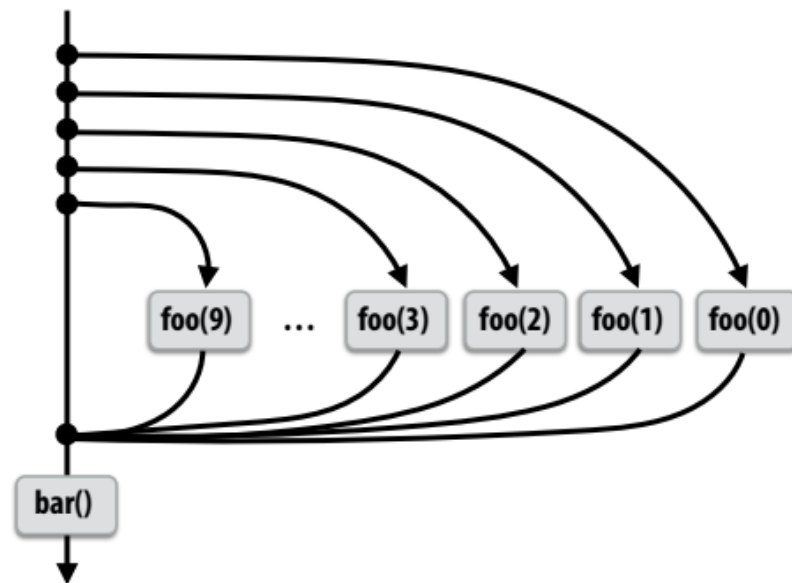
디스크립터는 블록의 미결 스폰 수와 완료된 스폰의 수를 추적함

디스크립터가 추적하는 1개의 스폰은 스레드 0에서 실행 중인 `foo(0)`에 해당합니다. (steal 후 continuation은 이제 스레드 1이 소유하고 있으므로).

Implementing sync: stealing case

block (id: A)

```
for (int i=0; i<10; i++) {  
    cilk_spawn foo(i);  
}  
cilk_sync; 블록 A 내에서 spawn된 모든 호출에 대한 동기화  
bar();
```



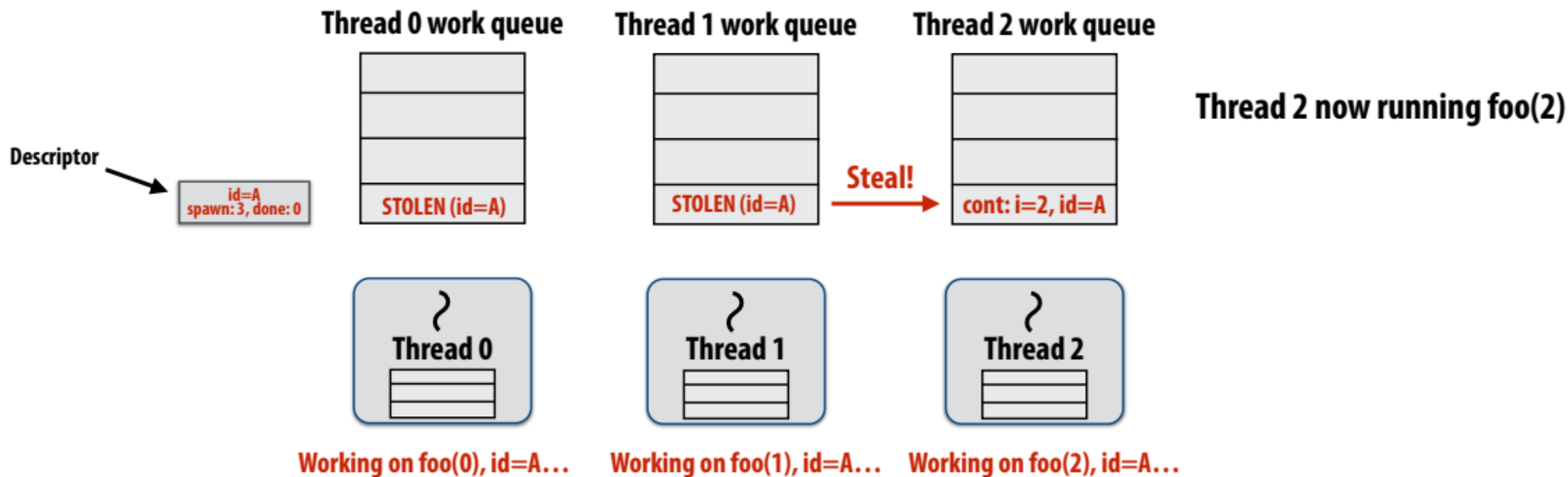
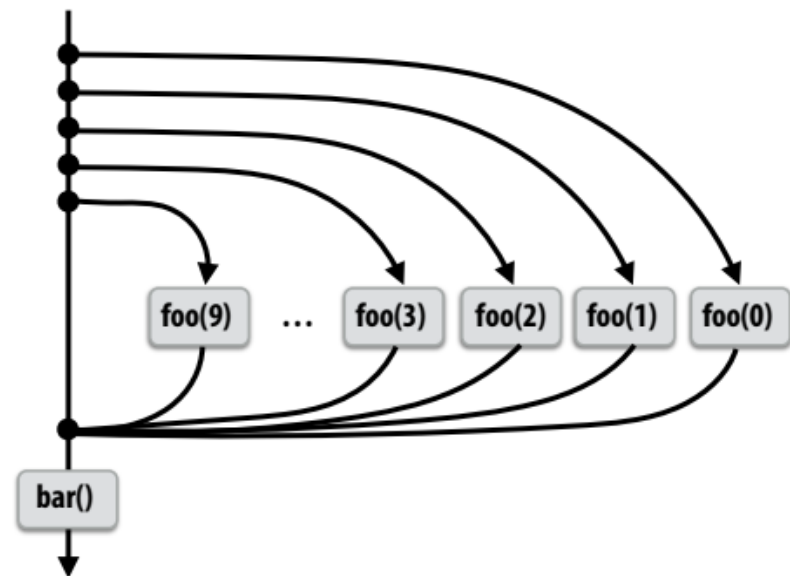
Thread 1 is now running foo(1)

Note: spawn count is now 2

Implementing sync: stealing case

```
block (id: A)
[
  for (int i=0; i<10; i++) {
    cilk_spawn foo(i);
  }
  cilk_sync;
  bar();
]
```

블록 A 내에서 spawn된 모든 호출에 대한 동기화

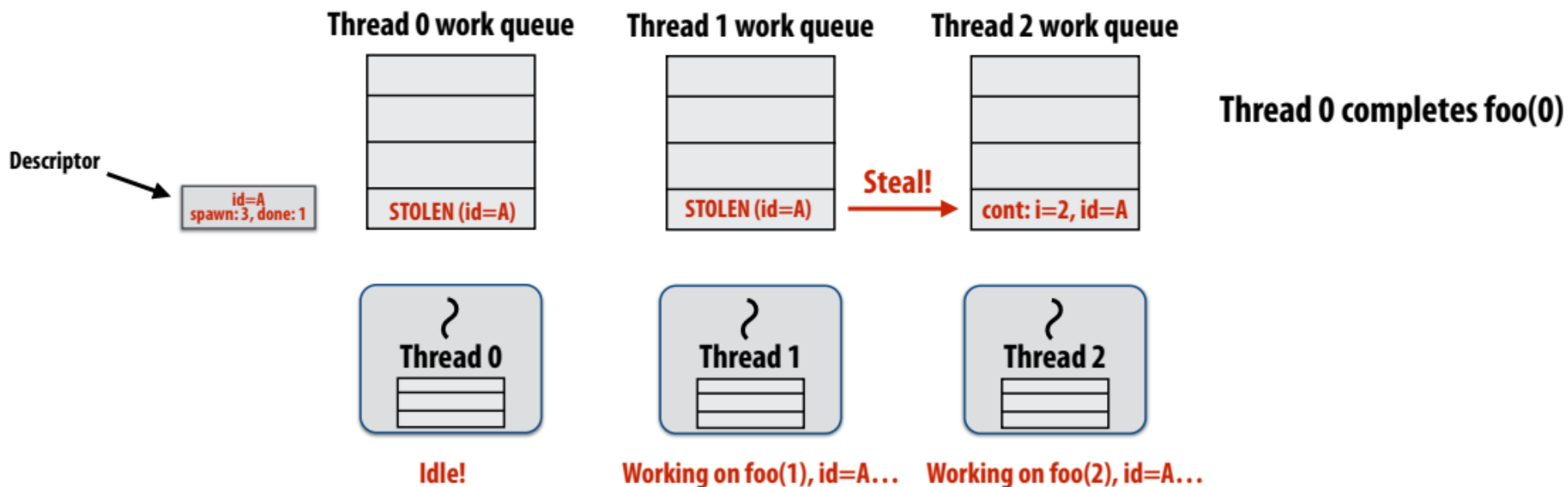
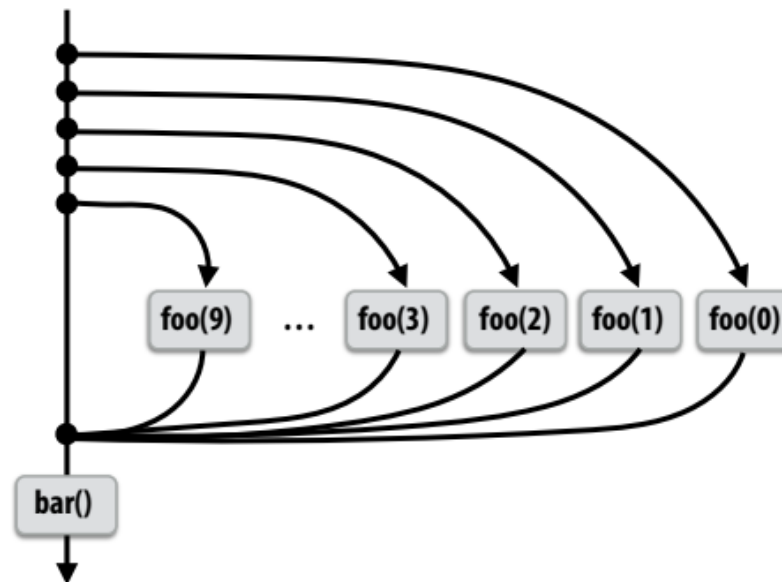


Implementing sync: stealing case

block (id: A)

```
for (int i=0; i<10; i++) {  
    cilk_spawn foo(i);  
}
```

```
cilk_sync; 블록 A 내에서 spawn된 모든 호출에 대한 동기화  
bar();
```

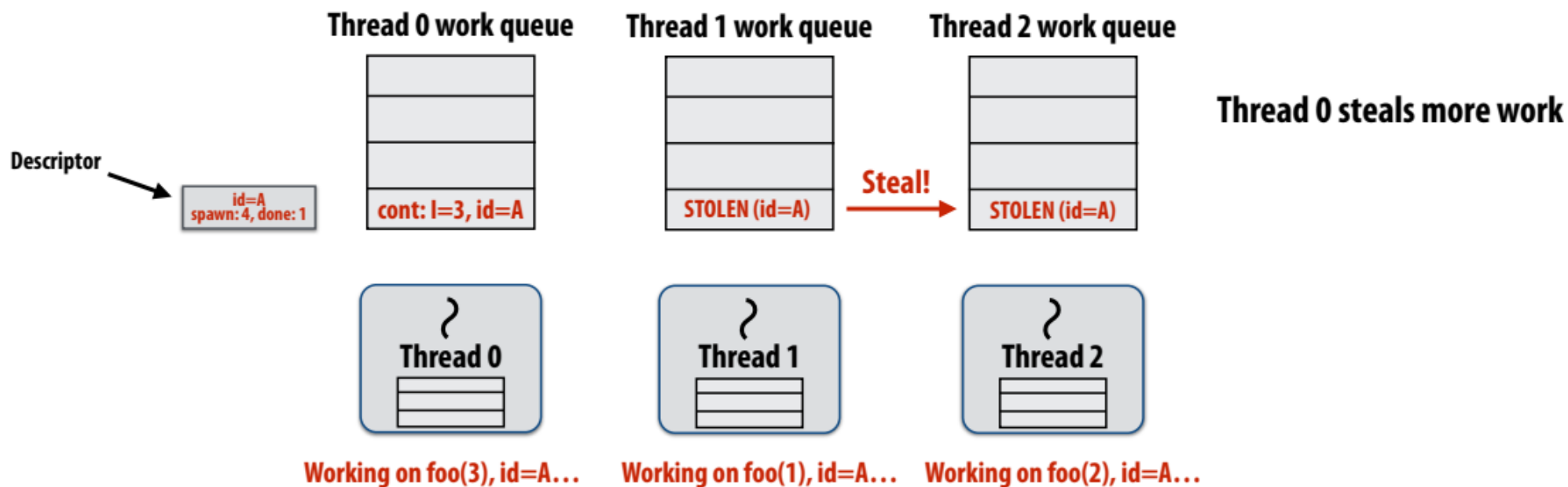
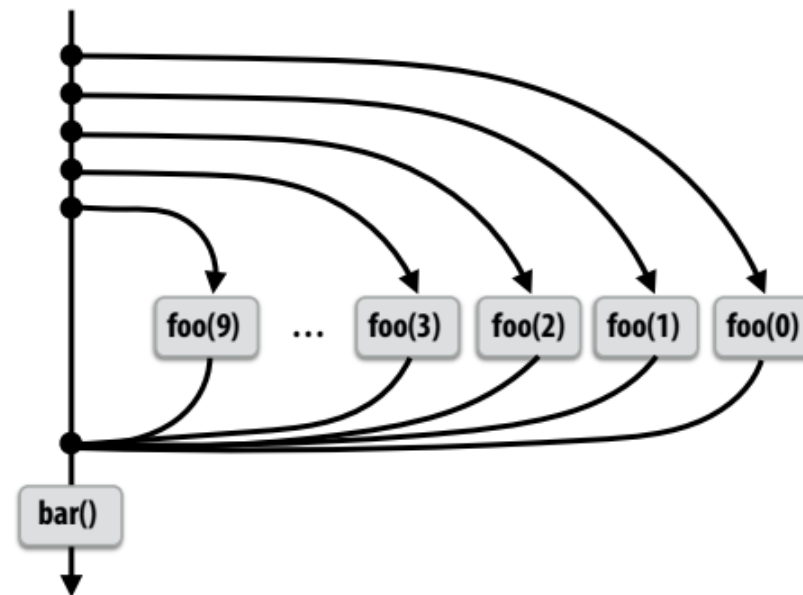


Implementing sync: stealing case

block (id: A)

```
for (int i=0; i<10; i++) {  
    cilk_spawn foo(i);  
}
```

```
cilk_sync;  블록 A 내에서 spawn된 모든 호출에 대한 동기화  
bar();
```

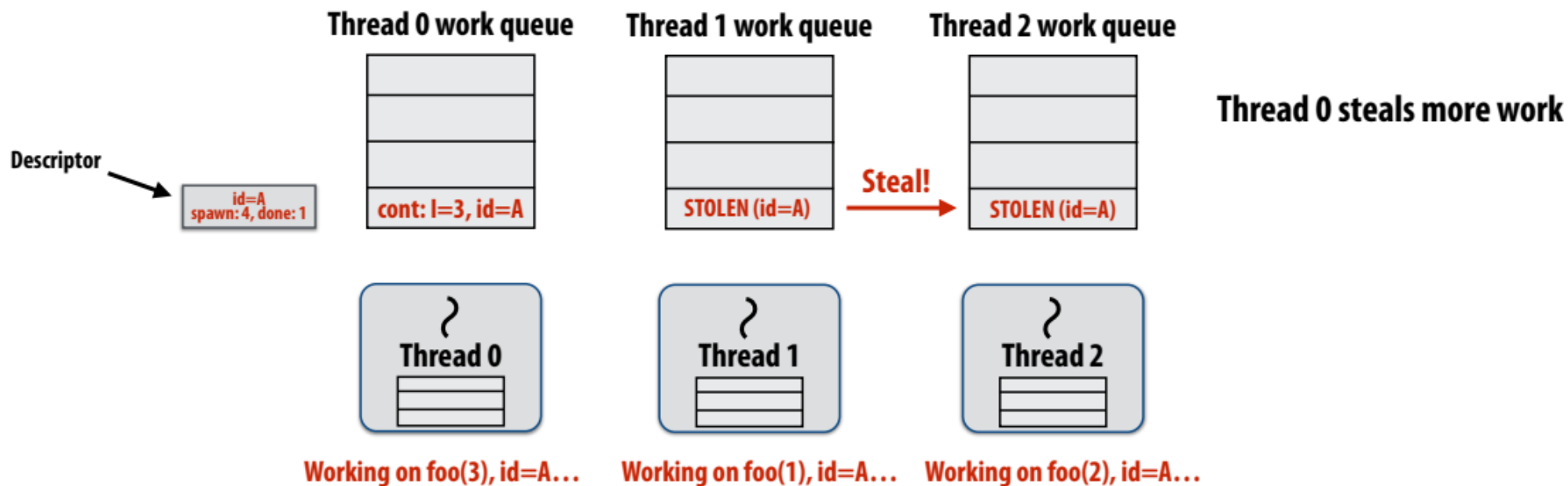
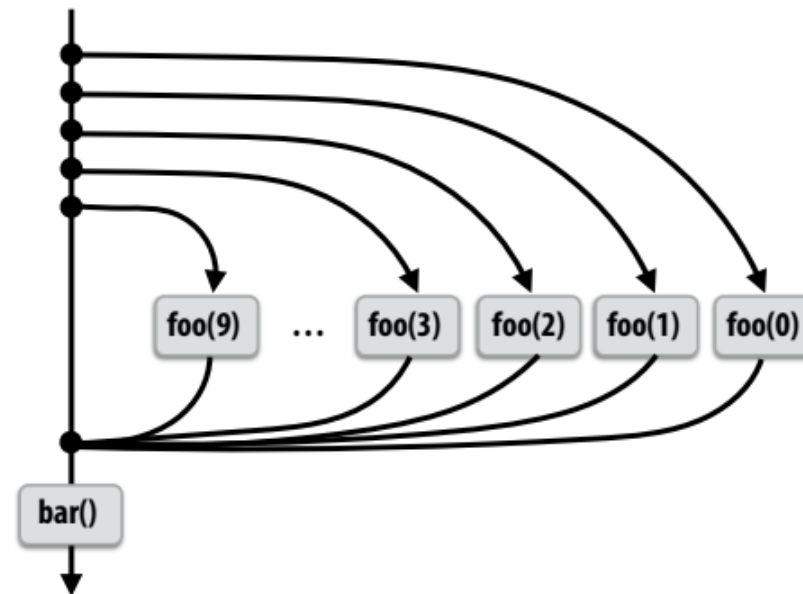


Implementing sync: stealing case

block (id: A)

```
for (int i=0; i<10; i++) {  
    cilk_spawn foo(i);  
}
```

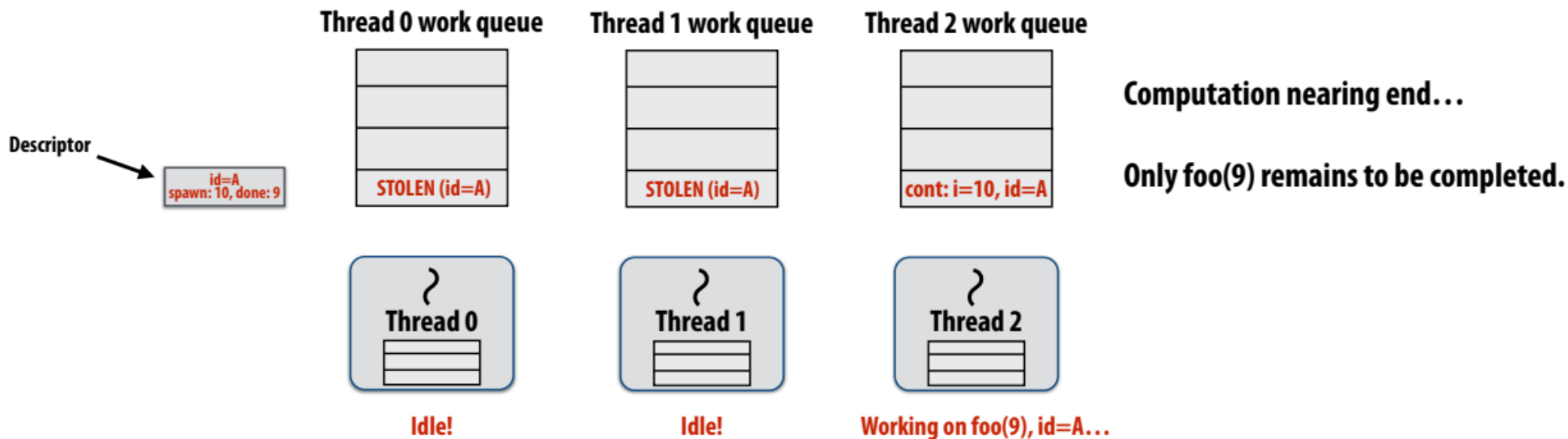
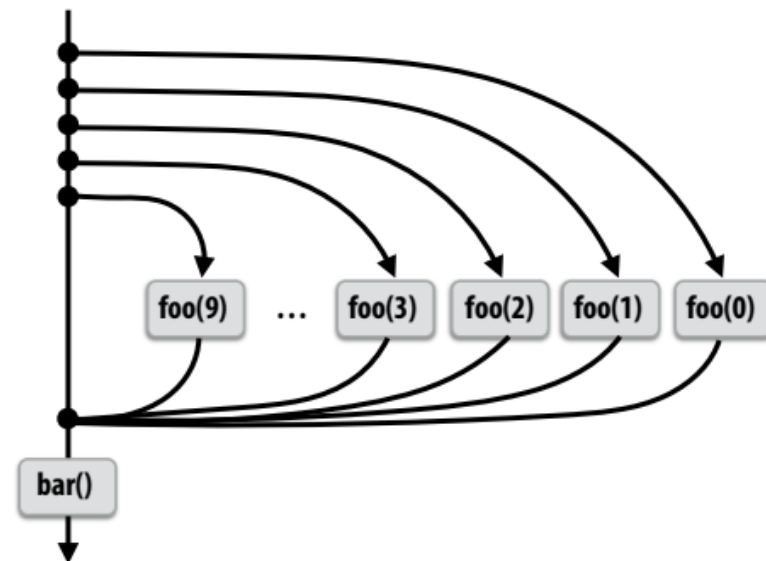
```
cilk_sync;  블록 A 내에서 spawn된 모든 호출에 대한 동기화  
bar();
```



Implementing sync: stealing case

```
block(id: A)
{
  for (int i=0; i<10; i++) {
    cilk_spawn foo(i);
  }
  cilk_sync;
  bar();
}
```

블록 A 내에서 spawn된 모든 호출에 대한 동기화

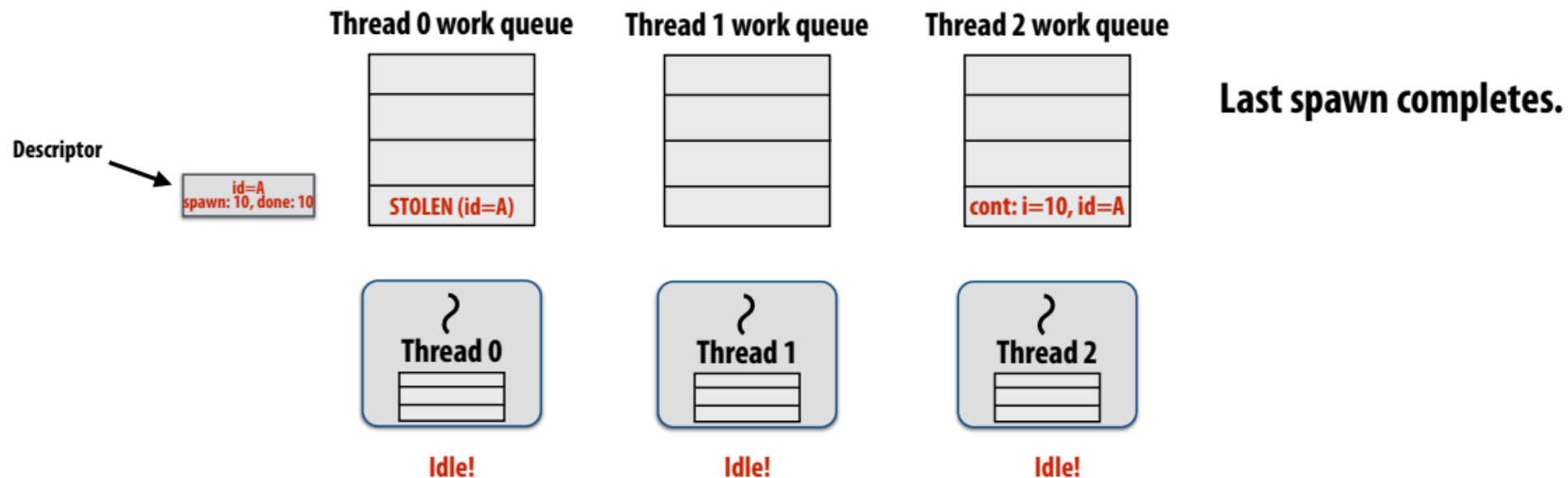
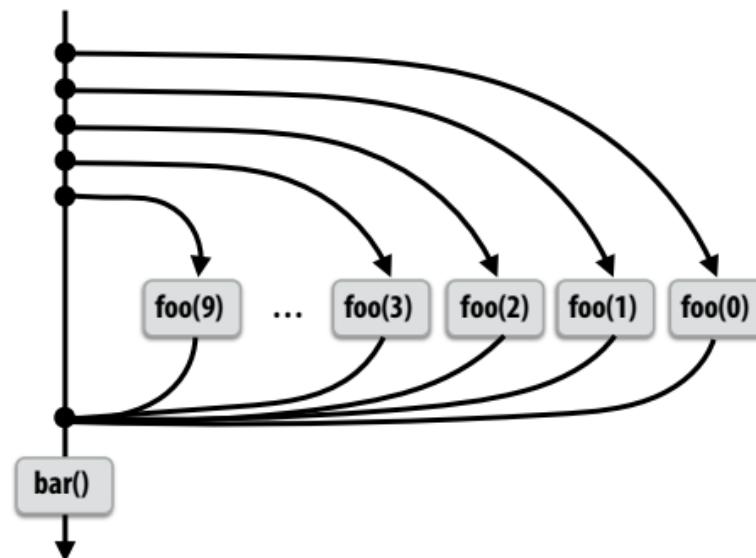


Implementing sync: stealing case

block (id: A)

```
for (int i=0; i<10; i++) {  
    cilk_spawn foo(i);  
}
```

cilk_sync; 블록 A 내에서 spawn된 모든 호출에 대한 동기화
bar();

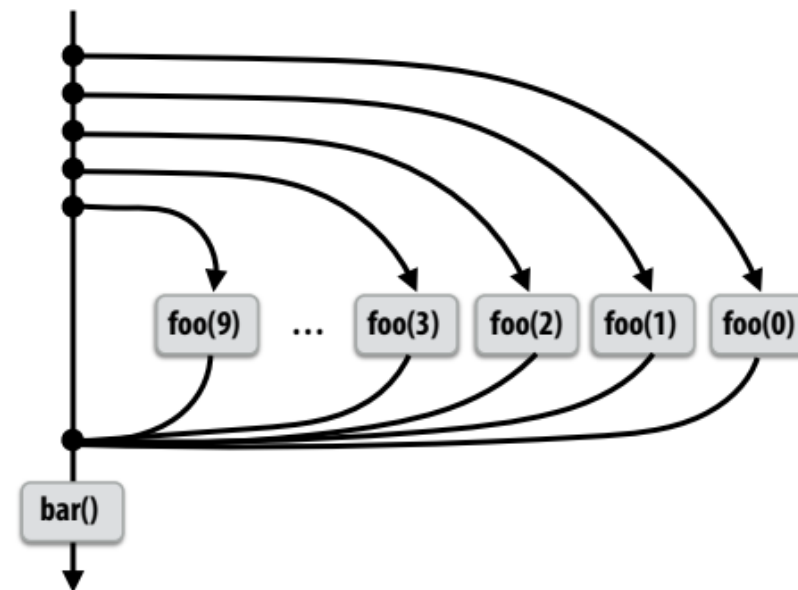


Implementing sync: stealing case

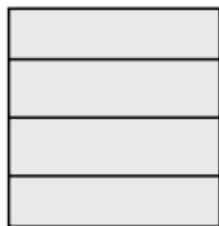
block (id: A)

```
for (int i=0; i<10; i++) {  
    cilk_spawn foo(i);  
}
```

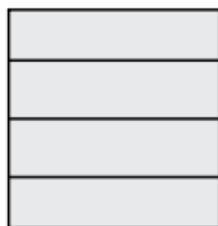
```
cilk_sync;  블록 A 내에서 spawn된 모든 호출에 대한 동기화  
bar();
```



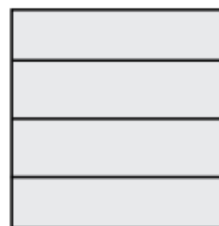
Thread 0 work queue



Thread 1 work queue



Thread 2 work queue



Idle!



Idle!



Working on bar()...

- 스레드 2가 이제 continuation을 재개하고 `bar()`를 실행.
- 참고 블록 A 설명자가 이제 자유.

Cilk uses greedy join scheduling

- Greedy join scheduling policy

- 할 일이 없으면 모든 스레드가 항상 훔치기를 시도.
- 스레드는 시스템에 훔칠 작업이 없는 경우에만 유휴 상태가 됨.
- 스폰을 시작한 워커 스레드는 `cilk_sync` 이후 로직을 실행하는 스레드가 아닐 수 있음.

- Remember:

- steal을 기록하고 동기화 지점을 관리하는 오버헤드는 steals이 발생할 때만 발생.
- 대량의 작업이 steal되는 경우, 이는 드물게 발생되어야 함.
- 대부분의 경우, 스레드는 local deque에서 local work을 푸시/팝업.

- **Fork-join parallelism: 분할-정복 알고리즘을 표현하는 자연스러운 방법**
 - Cilk Plus에 대해 논의했지만, 다른 많은 시스템에도 포크/조인 프리미티브가 있음(예: OpenMP).
- **locality-aware work stealing scheduler로 spawn/sync abstraction를 구현하는 Cilk Plus 런타임**
 - 항상 스폰된 자식 실행(continuation stealing)
 - 조인 시 탐욕스러운 동작(스레드가 조인 시 대기하지 않고 즉시 훔칠 다른 작업을 찾음)